

온습도 변화에 강인한 저비용 교량 진동 이상 탐지 시스템

1 주제

1.1 개발 배경 및 필요성

노후 교량의 갑작스러운 붕괴 - 반복되는 사회 문제

2023년 4월, 준공 약 30년의 분당 정자교 보행로가 캔틸레버 부재의 부식·손상 누적으로 일상 보행 중 붕괴해 1명이 사망하고 1명이 다쳤다. 2022년 1월 미국 피츠버그 Fern Hollow Bridge도 부식이 점검에서 지적되고도 보수가 지연되다 출근 시간대에 붕괴했다. 두 사고 모두 점검에서 징후가 관측되고도 보수·보강으로 이어지지 못한 사례로, 정기 점검만으로는 구조물 상태 변화를 연속적으로 파악해 사고를 막기 어렵다는 점을 보여준다.



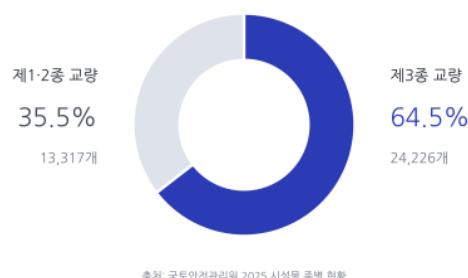
[자료1: 정자교 붕괴 현장 사진]



[자료2: Fern Hollow Bridge 붕괴]

두 사례처럼 노후 교량에서 장기간 진행되는 부식·균열·피로 누적은 강성·감쇠·고유진동수 같은 동특성 변화로 이어지므로 진동 기반 모니터링으로 관측할 수 있다. 그러나 진동 특성은 손상뿐 아니라 온도·습도 같은 환경 변수로도 변해, 관측된 변화가 손상 탓인지 환경 탓인지 구분하기 어렵다. 결국 점검과 점검 사이의 동특성 변화를 연속적으로 포착하는 일, 그리고 그 변화에서 환경의 영향을 분리해 손상에 의한 이상 징후만 가려내는 일이 과제로 남는다.

국토안전관리원 2025년 시설물 중별(규모에 따른 분류) 현황 기준 전국 교량 시설물은 제1종 5,392개, 제2종 7,925개, 제3종 24,226개로 총 37,543개이며, 상대적으로 규모가 작은 제3종 교량이 약 64.5%를 차지한다. 그만큼 안전관리 대상이 넓어, 중소형 교량에도 적용 가능한 저비용 상시 스크리닝 기술의 필요성이 커지고 있다.



[자료3: 전국 교량 중 제3종 교량의 비율]

중소형 교량의 상시 모니터링 사각지대

한편 교량의 상태는 규모와 별개로 안전등급(A~E)으로 평가되며, 2025년 12월부터는 노후·저등급(C·D·E) 시설물까지 정밀안전진단 대상이 확대되었다. 그러나 정밀안전진단은 비용·인력 부담이 크고 안전등급에 따라 4~6년 등 일정 주기로만 시행되어, 점검과 점검 사이에는 시간적 공백이 남고 그사이 동특성 변화가 누적되어도

감지하기 어렵다. 결국 보행자 육교·소형 보도교 등 노후 중소형 교량에는 점검 사이 상태 변화를 상시 감지할 저비용 보조 모니터링 수단이 부족하다.

종류\안전등급	A등급	B·C등급	D·E등급
정기안전점검	반기 1회 이상	반기 1회 이상	1년 3회 이상
정밀안전진단	6년 1회 이상	5년 1회 이상	4년 1회 이상

[자료4: 시설물 등급별 점검 주기 및 대상 표]

정밀 진단 이전 단계의 저비용 스크리닝 기술 부재

기존 SHM(Structural Health Monitoring) 시스템은 센서를 구조물에 상시 설치해 연속 계측하는 기술이나, 초정밀 압전형 가속도계, 고성능 수집 장비, 광섬유 분포형 센서(KRIS BOCDA 등) 같은 정밀 계측 장비와 전용 인프라에 의존해 인천대교와 같이 규모와 중요도가 큰 일부 대형 교량 위주로 적용되며, 운영 중인 중소형 노후 교량에 저비용으로 보편 적용하기에는 경제성·시공성 제약이 크다. 그 결과 주기적 정밀 진단과 고비용 상시 계측 사이, 즉 정밀 진단 이전 단계의 저비용 상시 스크리닝 기술은 사실상 비어 있다. 따라서 교량 안전관리는 상시 스크리닝(1차)으로 이상 후보를 걸러낸 뒤 정밀안전점검·정밀안전진단으로 이어지는 다단계 개념으로 보완될 필요가 있으며, 그래야 한정된 점검 인력과 예산을 의심 교량에 집중할 수 있다.

1.2 개발의 주제 및 목적

본 작품은 앞서 정리한 세 문제, 즉 ① 점검 사이 동특성 변화의 연속적 포착과 환경 영향의 분리, ② 정밀 진단 주기 사이의 시간적 공백, ③ 정밀 계측 시스템의 중소형 노후 교량 적용 곤란의 해결을 목표로 한다. 붕괴를 예측하거나 정밀안전진단을 대체하는 시스템이 아니라, 정밀 점검의 우선순위를 제시해 점검과 진단 사이의 시간적 공백을 보완하는 저비용 1차 스크리닝(보조) 시스템이다.

본 작품의 목적은 이 사각지대와 기술 부재를 동시에 해결하는 저비용 머신러닝 기반 교량 진동 스크리닝 시스템의 개발이며, 목표는 세 가지다. 첫째, ADXL355Z MEMS 가속도계와 Raspberry Pi(1차 3 Model A+, 2차 5)로 진동 데이터를 수집하되, 운영 중인 교량에서는 손상 데이터를 일부러 만들 수 없으므로 정상 데이터만으로 학습하는 비지도 학습을 택한다. 둘째, 교량 고유진동수는 손상이 없어도 기온·습도에 따라 오르내리므로[2], DHT22로 온도·상대습도를 함께 기록하고 정상 상태의 온습도-고유진동수 관계를 학습해 환경으로 설명되는 변동을 걸러내 거짓 경보를 줄인다. 셋째, 이상 신호만으로는 점검자가 어디를 살펴야 할지 알기 어려우므로, 어느 주파수 대역이 정상 기준을 벗어났는지 함께 제시해 후속 정밀 점검의 방향을 잡아 주는 설명 가능한 알람을 구현한다.

해결 방법

세 목표는 하나의 비지도 이상 탐지 파이프라인으로 구현한다. 1) 정상 상태의 진동 패턴만 학습해 두고, 2) 온습도로 설명되는 정상 변동을 먼저 걸러낸 잔차가 학습된 패턴에서 크게 벗어나면 이상 후보로 판단하며, 3) 이때 어느 주파수 대역의 재구성 오차(모델이 학습한 정상 패턴으로 입력을 복원했을 때 원본과의 차이로, 정상에서 벗어날수록 커진다 — 원리는 2.3)가 큰 지 시각화해 확정 진단 대신 점검 방향 판단을 돕는 해석 정보를 함께 제공한다. 하드웨어 구성은 2.2, 세 기능을 연결한 구체적 절차는 2.3 작동 원리에서 단계별로 다룬다.

1.3 유사 제품(연구) 및 차별점

저비용 MEMS 가속도 센서 기반 SHM 시스템

Raspberry Pi와 저가형 MEMS 가속도계(ADXL355)를 조합해 수천만 원대 초정밀 압전형 센서 없이도 구조물의 고유진동수·모드 형상·감쇠 특성을 식별할 수 있음을 입증한 SHM 연구가 있다.[1] 다만 이 연구는 동적 특성 식별에 초점을 두어, 온습도 등 환경 변수나 운영 하중 변화가 이상 탐지에 미치는 영향은 다루지 않는다.

광섬유 분포형 센서 시스템

한국표준과학연구원(KRISS)의 BOCDA 기반 광섬유 센서 시스템은 구조물의 변형률과 하중 위치를 높은 공간 해상도로 정밀 계측하는 기술로, [4] 전용 인프라를 전제로 한 정밀 변형률 계측이라는 점에서 측정 물리량과 적용 목적이 본 작품의 진동 기반 상시 스크리닝과 다르다.

비지도 학습 기반 SHM 알고리즘

스위스 Z24 Bridge Benchmark(KU Leuven 제공)는 장기 모니터링과 점진적 손상 시험 데이터를 포함한 대표적 공개 데이터셋이다. [3] Autoencoder, One-Class SVM, Isolation Forest, VAE 등 다양한 비지도 학습 모델이 Z24에서 비교되어 왔으나 [5][6], 대부분 이상 점수나 임계값 초과 여부 출력에 그쳐 어느 대역이 기준을 벗어났는지는 설명하지 않는다.

실규모 교량 장기 모니터링 데이터 기반 환경 보정 연구

독일 바우첸(Bautzen)의 실규모 연구용 교량을 장기간 모니터링한 openLAB Research Bridge 공개 데이터셋은 가속도 응답과 기온·습도 등 환경 측정값을 함께 제공해 계절 변화·일교차의 영향 분석에 적합하다(상세는 2.7 참조).[8] 본 작품에서 openLAB은 환경 보정 원리가 실규모 장기 데이터에서도 타당한지 확인하는 보조 검증 데이터이며, 기여의 중심은 수집·보정·판단·설명에 한 파이프라인으로 연결한 저비용 하드웨어 구현과 자체 모형 교량 실측 검증에 있어 공개 데이터 재분석 연구와 구별된다.

따라서 이 작품의 차별성은 저비용 하드웨어 직접 구현, 온습도 환경 보정, 대역별 설명 가능성의 세 축으로 정리된다. 즉 본 작품은 저비용 MEMS 연구와 달리 온습도 보정으로 거짓 경보를 줄이고, 광섬유 계측과 달리 저가 범용 부품으로 상시 스크리닝을 수행하며, 이상 점수만 출력하는 비지도 학습 연구와 달리 대역별 재구성 오차 기여도까지 제시한다.

2 작품 제작

2.1 목표

이 시스템은 저비용·저전력 장치와 비지도 학습을 기반으로 노후 중소형 교량의 진동 이상 후보를 조기에 감지하기 위한 보조용 진동 스크리닝 시스템이다. 본 시스템은 ADXL355Z 기반 MEMS 가속도계와 DHT22 온습도 센서를 이용해 진동, 온도, 상대습도 데이터를 함께 수집하고, 정상 상태에서 학습한 온도·습도·진동수 관계를 이용해 환경 변화에 따른 거짓 경보를 줄이는 것을 목표로 한다. 이후 Autoencoder 기반 비지도 학습으로 정상 진동 패턴에서 벗어난 이상 후보를 탐지하며, 이상 발생 시 어느 주파수 대역 또는 모드 주변 대역의 재구성 오차가 큰지 함께 제시하여 정밀 점검 우선순위 판단을 돕는다.

본 작품의 자체 실험 검증은 두 단계로, 1차 실험은 폼보드 모형과 수동 가진으로 수집·보정·탐지·설명 파이프라인 전체의 동작을 확인하고, 2차 실험은 1차의 두 한계(공진 피크 불선명, 수동 가진 편차)를 알루미늄 다경간 모형과

솔레노이드 오토 태퍼로 개선해 온도·습도·질량·볼트 플림 조건 매트릭스에서 구분 성능을 검증하며, Raspberry Pi 5 기반 실시간 측정·분석 체계를 함께 구축한다(상세는 2.2·3.1 참조).

2.2 작품 설계

하드웨어 구성

본 작품은 하드웨어 수집부와 소프트웨어 분석부로 나누어 제작하며, 하드웨어 구성 요소와 설계 이유는 다음 표와 같다.

구성 요소	역할·설계 이유
ADXL355Z 3축 가속도계	3축 진동 측정 — 고유진동수 등 진동 특성 변화 감지
DHT22 온습도 센서[7]	온습도 측정 — 환경 기인 정상 변동 보정
Raspberry Pi 3 Model A+ (1차)	데이터 수집·저장 (Python 로깅)
Raspberry Pi 5 (2차)	실시간 수집·태퍼 제어·FFT·이상 점수·대시보드를 한 장치에서 실행
폼보드 교량 모형 (1차)	질량 분포 등 구조 조건 실험용 1차 모형
알루미늄 교량 모형 (2차)	감쇠가 작아 공진 피크 선명, 볼트 체결로 경계조건 반복성 확보
솔레노이드 오토 태퍼 (2차)	일정 세기·주기·위치 자동 가진 — 수동 편차 제거로 정상 산포 축소

핵심 계측·처리 장치의 비용은 ADXL355Z 평가보드 84,589원, DHT22 온습도 센서 5,680원, Raspberry Pi 5(RAM 2~4 GB 모델 기준 약 100,000원)로 합계 약 190,000원(19만 원대)이다. 초정밀 압전형 가속도계와 전용 수집 장비를 전제로 하는 기존 SHM 계측(수천만 원대 압전형 센서 기반, 1.3·[1] 참조) 대비 크게 낮은 수준으로, 중소형 교량 다수에 보급 가능한 저비용 구성이라는 본 작품의 목표를 뒷받침한다.

소프트웨어 구성

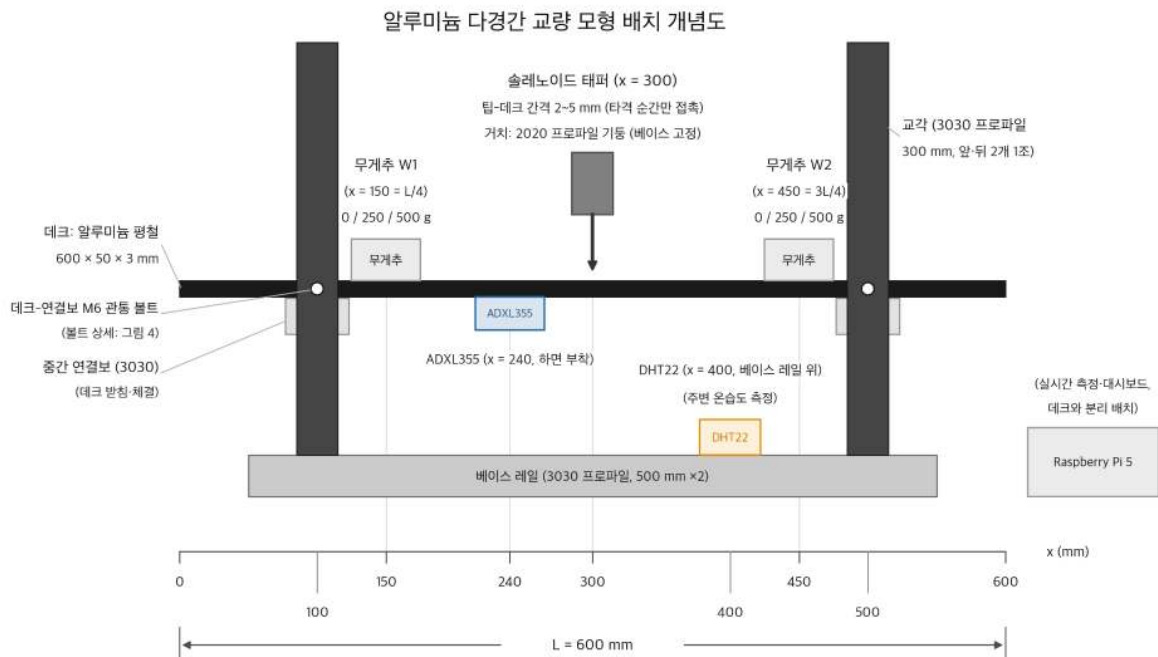
소프트웨어 분석부는 수집·분석 두 프로그램으로 구성한다. 수집 프로그램은 Raspberry Pi에서 Python으로 동작하며 ADXL355Z 3축 가속도를 200 Hz(2차 실험은 500 Hz)로, DHT22 온도·상대습도를 약 2초 간격으로 한 CSV 파일에 기록한다. 분석 프로그램은 이 파일을 시간 창(window) 단위로 나눠 FFT·피쳐 추출, 온도 T·상대습도 RH 다변량 회귀 잔차 계산, 표준화, 재구성 오차 기반 이상 점수 산출, 99번째 백분위수 임계값 판정, 대역별 기여도 출력 순으로 동작한다.

2차 실험에서는 이 구성을 Raspberry Pi 5 기반의 실시간 시스템으로 확장한다. 수집 로거·태퍼 제어·분석 서버를 같은 장치에서 실행해 동일 시계를 보장하고, 타격 직후 창(타격 시각 t_0 기준 $t_0+0.05$ s부터 2.0 s 동안)을 잘라 스펙트럼과 이상 점수를 즉시 계산해 웹 대시보드로 표시한다. 대시보드는 실시간 z축 파형·타격 스펙트럼·임계값 대비 배율(이상 점수 ÷ 임계값)·피크 주파수·RMS·온습도·대역별 기여도를 약 0.7초 주기로 갱신하며, 배율 0.8 미만은 정상, 0.8~1.0은 주의, 1.0 이상은 이상 후보 상태로 표시한다. 실시간 판정은 현장 모니터링용이고, 원본은 조건 코드를 포함한 CSV로 전부 저장해 모델 학습·평가에 사용한다.

센서 부착 위치

1차 폼보드 모형에서 ADXL355Z는 1차 휨 응답이 최대인 중앙부(경간 L·폭 B 기준 $x = L/2$, $y = B/2$)에 부착하고, 상하 휨을 반영하는 수직 z축을 주요 해석 대상으로 삼는다.

2차 알루미늄 모형에서는 데크 좌측 끝단을 $x = 0$ 으로 하는 길이 방향 좌표(데크 전장 600 mm)로 모든 위치를 정의한다. 솔레노이드 태퍼의 타격점은 중앙 $x = 300$ mm 상면 폭 방향 중앙으로 하여 매 타격이 1차 모드를 일관되게 가진하도록 하고, 팁과 데크 사이 간격은 카드 게이지로 2~5 mm를 전 조건 동일하게 유지해 타격 순간에만 접촉하도록 한다. ADXL355Z는 타격점 직하부를 피한 $x = 240$ mm의 데크 하면에 부착해, 타격 직후의 국부 충격 성분과 상면 무게추·타격점과의 간섭을 피하면서 중앙 인근의 큰 휨 응답을 측정한다. 무게추는 1차 실험과 같은 비율인 $x = 150$ mm($L/4$)와 $x = 450$ mm($3L/4$) 두 지점에 위치시킨다. DHT22와 Raspberry Pi는 두 실험 모두 교량의 질량·강성에 영향을 주지 않도록 진동 구조(데크·교각) 밖에 둔다. 1차에서는 모형 옆에 두고 케이블을 여유 있게 고정했으며, 2차에서는 DHT22를 베이스 레일 위 $x = 400$ mm 지점에 고정해 교량 주변의 온도·상대습도를 전 조건 동일한 위치에서 기록하고, Raspberry Pi는 데크에서 떨어진 베이스 옆에 두어 배선 장력과 질량이 진동 응답에 영향을 주지 않도록 한다.



[그림1: 알루미늄 다경간 교량 모형 배치 개념도(2차 실험 기준)]

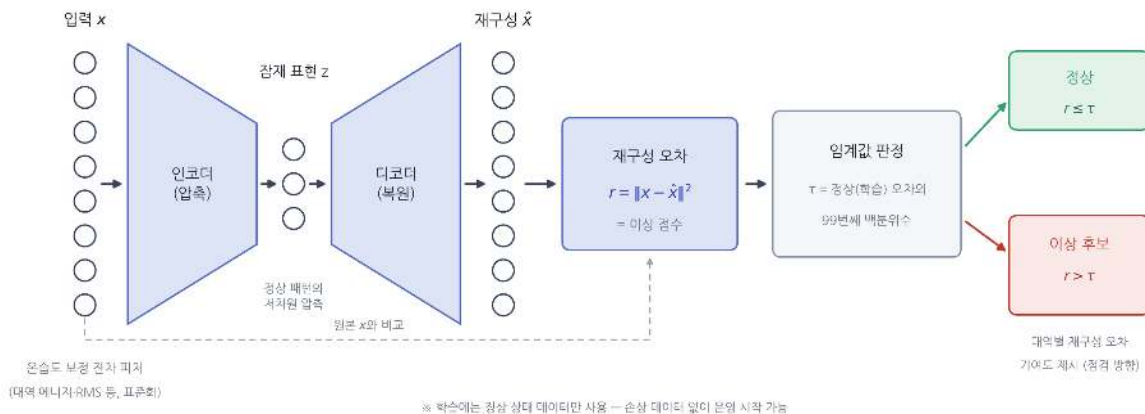
2.3 작동 원리

- 진동 및 온습도 데이터 수집:** ADXL355Z에서 3축 가속도를, DHT22에서 온도 T 와 상대습도 RH 를 수집한다. 예비 실험으로 실제 고유진동수 범위를 확인해 분석 대역을 나이퀴스트 한계 이내에서 설정하며, 실측 1차 고유진동수가 200 Hz의 나이퀴스트 한계(100 Hz)에 근접·초과할 경우 샘플링 주파수를 500 Hz로 상향하도록 계획하였다. 2차 알루미늄 모형은 조립 후 예비 측정에서 1차 고유진동수가 이 한계를 넘어, 샘플링 주파수를 500 Hz로, 측정 레인지를 ± 8 g로 상향 확정하였다(구체적 실측 고유진동수는 3.4에 기입). 가진은 1차 수동 타격, 2차 솔레노이드 오토 태퍼의 자동 타격(위밍업 후 일정 주기)으로 수행하고, 타격 시각을 동일 시계로 기록해 타격 직후의 자유진동 구간만 분석 단위로 사용한다.
- 신호 전처리와 주파수 대역 피쳐 추출:** 가속도 신호에서 DC offset을 제거하고 스펙트럼을 추정한다(1차 FFT/STFT, 2차는 신호를 짧은 구간으로 나눠 스펙트럼을 평균함으로써 잡음 변동을 줄이는 Welch 방법). 구간·축별로 RMS, 표준편차, peak-to-peak, 피크 주파수·파워, 대역별 에너지 등을 계산하며, 분석 대역은 실측 1차 고유진동수에 맞춰 재설정한다(1차: 예비 피크 22.6 Hz 기준 0~12·12~25·25~50 Hz, 2차: 실측 101 Hz 기준 0~40·40~80·80~125 Hz — 3.4 참조). 2차 확정 구현은 z축 8개 피쳐(로그 RMS·로그 p2p·피크

주파수·로그 피크 파워·로그 총에너지·3개 대역 에너지 비율)로, 진폭·에너지 계열을 로그 스케일로, 대역 에너지를 전체 대비 비율로 정규화한다(근거는 3.5 참조). 온도 T·상대습도 RH는 환경 보정에 사용한다. z축 피크 주파수를 1차 고유진동수의 지표로 삼으며, 대역별 분석은 손상 위치 특성이 아니라 어떤 진동 특성이 기준을 벗어났는지 설명하는 보조 지표다.

3. 온도·습도 기반 정상 고유진동수 다변량 회귀 보정: 정상 데이터에서 온도 T와 상대습도 RH를 설명 변수로 1차 고유진동수를 예측하는 다변량 회귀를 학습하고, 추론 시 예측값 $\hat{f} = g(T, RH)$ 와 실측값 f 의 차이 $r = f - \hat{f}$ 를 온습도 효과를 보정한 잔차로 정의한다. 같은 회귀·잔차 계산을 각 진동 피처에 적용해 얻은 잔차 피처들을 이상 탐지 모델의 입력으로 쓴다. 잔차에는 구조적 이상 후보 외에 센서 노이즈·부착 상태·운영 하중의 영향도 남을 수 있어, 손상 확정값이 아니라 이상 후보 탐지 기준으로 사용한다.

4. Autoencoder 기반 비지도 학습 이상 탐지: 표준화된 정상 상태의 잔차 피처만으로 Autoencoder를 학습한다(구조는 그림2 참조). 모델은 입력을 저차원 잠재 표현으로 압축·복원하며 정상 데이터의 상관 구조만 학습하므로, 정상 패턴에서 벗어난 데이터는 재구성 오차가 커진다. 임계값은 정상 검증 데이터 재구성 오차의 99번째 백분위수로 설정하며(2차에서는 학습에 쓰지 않은 임계값 전용 정상 창 90개의 오차 분포 사용, 3.2 참조), 학습과 임계값 설정이 모두 정상 데이터만으로 이루어져 손상 데이터 없이 운영을 시작할 수 있다. 1차 구현은 선형 Autoencoder와 동일한 PCA 재구성 오차를 사용하였고, 2차에서는 은닉 8-3-8 구조의 비선형 Autoencoder(tanh, 조기 종료)를 채택하였다.



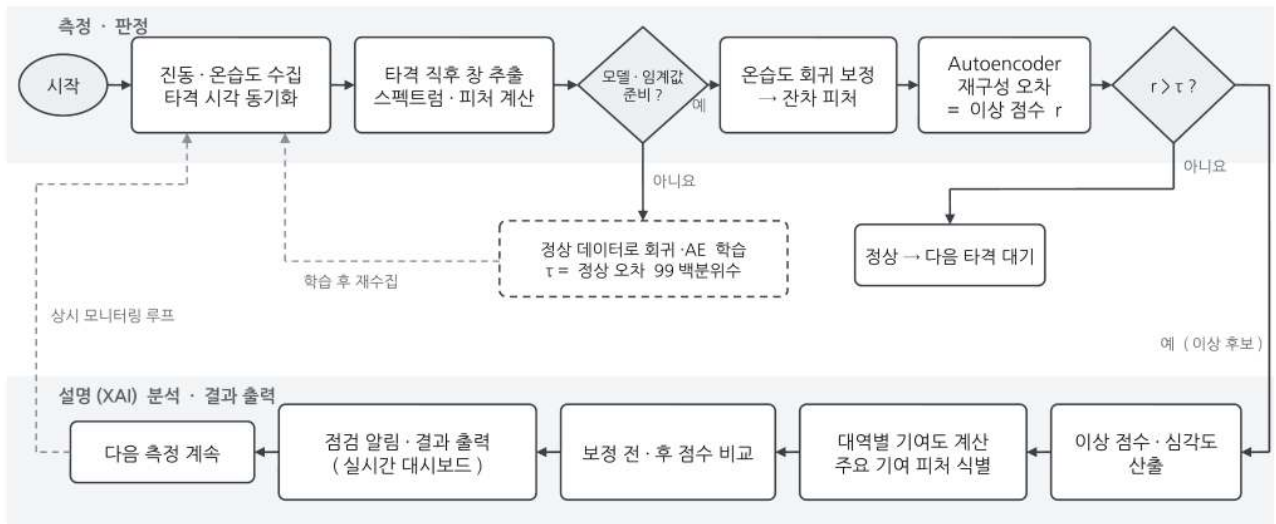
[그림2: Autoencoder 기반 이상 탐지 구조]

5. 대역별 재구성 오차 기여도 분석: 이상 감지 시 어느 주파수 대역(또는 모드 주변 대역)의 재구성 오차가 큰지 함께 분석한다. 기여도는 “해당 대역의 재구성 오차 ÷ 전체 재구성 오차”로 계산하며, SHAP은 보조 모델에 추가 적용 가능한 선택 기능으로 두고 기본 설명 방식은 대역별 기여도로 한다.

6. 결과 출력 및 정밀 점검 알림: 이상 점수가 임계값을 초과하면 알람과 함께 이상 점수, 온습도 보정 전후 비교, 주요 기여 대역, 1차 고유진동수 변화율, 대역별 기여도, 점검 참고 해석을 출력한다. 초과 여부(탐지)와 별도로 임계값 대비 배율(2.2)과 최근 구간의 임계값 초과 비율을 심각도 지표로 산출해 정밀 점검 우선순위 근거로 제시하되, 이 지표는 손상의 물리적 심각도가 아니라 정상 상태 대비 이탈 정도를 나타내므로 서로 다른 손상 유형 간 비교에는 사용하지 않는다(검증 계획은 3.3 참조).

2.4 시스템 순서도

위 모듈을 소프트웨어 단위로 연결한 작동 구조는 아래 시스템 순서도와 같다. 수집→전처리·피처 추출→환경 보정→이상 점수 산출까지는 매 분석 창마다 반복되는 상시 경로이고, 이상 점수가 임계값을 초과하는 경우에만 대역별 기여도 분석과 정밀 점검 알림 경로로 분기한다. 각 블록은 2.3의 1~6단계에 순서대로 대응한다.



[그림3: 교량 진동 이상 탐지 시스템 순서도]

2.5 제작 과정

단계	내용	산출물
1단계 (06/03)	센서(ADXL355Z·DHT22·Pi 3A+) 연결 확인, 폼보드 모형 제작·센서 위치 결정	연결 테스트 결과·부착 사진
2단계 (06/03~06/04)	정상(0-0)·질량 조건(1-0, 1-1) 데이터 수집(표기는 3.1 참조)	조건별 raw CSV
3단계 (06/04)	환경 조건 수집(야외 온도 제외, 3.2 참조)·9개 조건 정리	9개 조건 데이터셋(기준·습도·온습도 동시)
4단계 (06/04~06/05)	FFT/STFT 피쳐 추출, Autoencoder 학습 및 이상 점수 계산	피쳐 데이터셋·탐지 결과
5단계 (06/05~06/12)	대역 기여도 분석·1차 결과 정리	기여도 그래프·1차 보고서
6단계 (06/17~07/08)	2차 부품(알루미늄 모형·태퍼) 구매·수급	부품 일체
7단계 (07/11~07/12)	모형 조립(레일·교각·연결보·데크), 태퍼 회로·Pi 5 실시간 대시보드 구현	2차 모형·과정 사진
8단계 (07/13~07/18)	정상 기준 데이터 수집 및 27조건 매트릭스 실험	조건별 CSV·타격 기록
9단계 (07/19~07/26)	2차 결과 분석, 모델 비교 및 설명서 반영	결과 표·그래프·최종 설명서

2.6 제작 항목 및 핵심 구현

하드웨어 제작

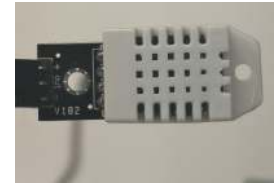
1차 실험에서는 가로 약 55 cm 폼보드 모형에 ADXL355Z·DHT22·Raspberry Pi 수집 체계를 구성해 온습도 보정 이상 탐지의 개념을 검증하였다. 다만 폼보드는 감쇠·조립 편차로 공진 피크가 완만해, 2차에서는 같은 센싱·보정 체계를 알루미늄 다경간 모형으로 확장하였다.



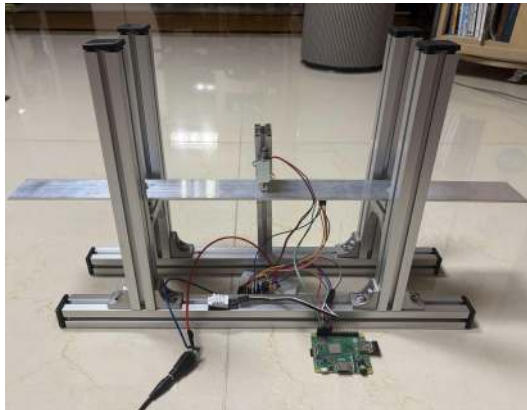
1차 실험용 폼보드 교량 모형 (가로 약 55 cm)



ADXL355Z 3축 MEMS
가속도계



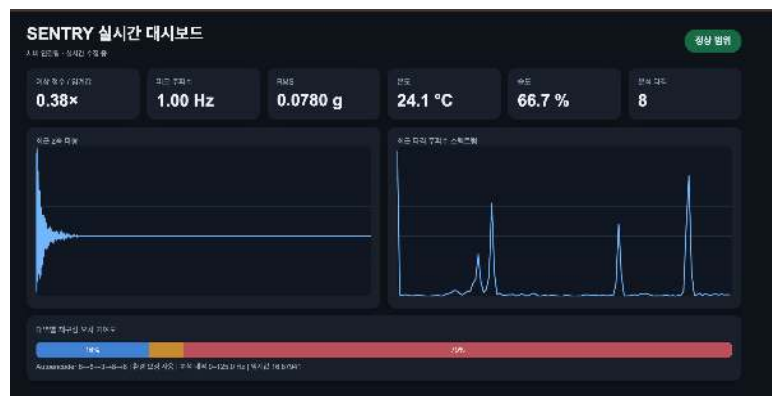
DHT22 온·습도 센서



2차 실험용 알루미늄 교량 모형(가로 약 60cm)



솔레노이드 오토 태퍼



Raspberry Pi 5 실시간 대시보드

2.7 openLAB 실규모 데이터 검증

제안한 파이프라인을 TU Dresden 연구 데이터 저장소(OPARA)로 공개된 openLAB 데이터셋[8]에 적용하였다. 대상은 총길이 약 45 m의 3경간 프리스트레스트 콘크리트 실규모 교량으로, 무손상 기준 상태에서 2024년 2월~10월 약 9개월간 가속도와 기온·습도가 함께 계측되어 계절 변화·일교차 변동이 누적되어 있고, 전 구간이 무손상이므로 임계값 초과 경보를 모두 거짓 경보로 해석해 보정에 따른 거짓 경보율(오탐율, False Alarm Rate) 변화를 정량 평가할 수 있다. openLAB의 가속도 응답은 진동 이벤트가 감지될 때마다 저장되는 이벤트 트리거 방식의 기록으로, 각 이벤트 기록을 하나의 분석 창으로 사용한다. 2.3과 동일하게 진동 피치를 추출하고 기온·습도를 시간 기준으로 연계한 뒤, 정상 표본을 시간 순서대로 학습용 263개·평가용 263개로 분할하고 학습 구간 재구성 오차의 99번째 백분위수를 임계값으로 무보정 AE, 온도 보정 AE, 온도·습도 보정 AE 세 모델을 동일 조건에서 비교하였다. 결과는 다음 표와 같다.

모델	학습 샘플	평가 샘플	학습 구간 이상 비율	평가 구간 오탐율
무보정 AE	263	263	0.011	0.019
온도 보정 AE	263	263	0.011	0.015
온도·습도 보정 AE	263	263	0.011	0.015

세 모델 모두 학습 구간 이상 비율은 0.011로 99번째 백분위수 임계값과 정합적이며, 무손상뿐인 평가 구간의 오탐율은 무보정 AE 1.90%, 두 보정 AE 모두 1.52%였다. 감소 폭 0.38%p(오탐 5→4건)는 1건 차이여서 단독으로는 통계적 근거가 되기 어려우나, 계절 변화·일교차가 누적된 9개월 실규모 데이터에서 보정 후 오탐율이 줄어드는 방향은 모형 실험(3.4)과 일관된다. 습도 추가로 더 줄지 않은 것은 변동이 주로 온도로 설명됨을 시사해 선행 연구[2]와 정합적이며, 습도 보정 자체의 효과는 습도 변화 구간 거짓 경보율이 17.5%에서 9.1%로 감소한 자체 모형 실험에서 확인된다. 이벤트 트리거 기록에 따른 가진원·진폭 변동은 세 모델에 공통 조건이므로, 보정 유무 간 오탐율 차이 해석에는 영향을 주지 않는다.

3 작품 테스트

3.1 테스트 목적 및 실험 설계

본 테스트의 목적은 두 가지이다. 첫째, 저비용 센서(2.2)와 모형 교량으로 구조 조건 변화가 진동 피치·이상 점수에 주는 영향을 확인한다. 1차는 파이프라인 동작을, 2차는 27조건 구분 성능을 검증한다. 둘째, 온습도 보정의 거짓 경보 감소를 검증한다. 실규모 비교는 openLAB(2.7)이 말고, 자체 실험은 같은 개선 경향의 재현을 확인한다.

무게추 기반 질량 분포 실험 설계

1차는 실제 손상 대신 지점당 100g(10g 무게추 10개)의 비파괴적 질량 분포 변화를 부여한다. 하중 지점은 센서가 있는 중앙부를 피해 L/4·3L/4로 두고, 우측 단일 하중은 좌측과 대칭에 가까워 1-0으로 대표한다. 조건 표기는 0-0(정상)·1-0(좌측)·1-1(양측)이다.

2차 실험 설계 — 알루미늄 다경간 모형과 27조건 매트릭스

아래 요인을 환경 3상태 × 질량 3수준 × 볼트 3수준 = 27조건 매트릭스로 수행한다. 환경은 전열기가 온도·습도를 동시에 바꿔 온도 단독 변화를 부여할 수 없다. 이에 변화없음(Tx-Hx)·습도변화(Tx-Ho)·온습도변화(To-Ho)의 세 상태로 구성한다. 이 설계는 환경(T·H)과 손상 모사(M·B)의 교호작용 데이터로, 환경 변화는 정상으로·손상은 이상으로 구분하는 온습도 보정의 가치를 검증한다.

요인(코드)	수준	부여 방법
환경 상태 (T·H)	변화없음 / 습도변화 / 온습도변화 (3수준)	변화없음 = Tx-Hx(환경 부여 없음). 습도변화 = Tx-Ho(불투명 박스+가습기로 고습 부여, 온도 유지). 온습도변화 = To-Ho(전열기 가열로 온도 상승·습도 동반 하강). 재현을 위해 전열기 거리·출력·가열 시간과 가습기 가동 시간을 기록하고, 분석에는 DHT22 실측값을 사용한다.
질량 변화 (M)	0 / 500 / 1,000 g (3수준)	x = 150 mm·450 mm 두 표식 위치를 사용해 M500은 한쪽에 500 g을 비대칭으로, M1000은 양쪽에 각 500 g을 대칭으로 부여. 보유 무게추 50 g×20개로 구성하며, 쌓는 순서와 방향까지 매번 동일하게 유지
볼트 풀림 (B)	0 / 1 / 2개 (3수준)	사전 지정한 데크-연결보 관통 볼트를 완전 조임에서 1/2바퀴씩 풀기. 1개 = BL1(좌측 타워, 비대칭 경계 손상 모사), 2개 = BL1·BL2(좌·우 타워, 대칭 경계 손상 모사)

풀림은 데크 경계조건에 직접 작용하는 데크-연결보 관통 볼트로 한정한다. B1이 B2의 부분집합이 되도록 구성해 경계 강성을 단계적으로 낮춘다. 연결보-교각-베이스-교각 볼트는 풀지 않는다. 볼트 조건 종료 후 원래 조임으로 복원해 정상 이상 점수가 기준선으로 돌아오는지 확인한다. 정상 클래스는 M0·B0인 3셀(환경 3상태)로 정의해 환경만 변한 상태를 정상으로 학습시킨다. 이 3셀은 학습·임계값 산정용이라 다른 조건보다 길게 수집한다.

실행은 환경 3블록으로 묶어 블록별 질량×볼트 9조건을 수행한다. 조건당 워밍업 10회(미수집) 뒤 본 타격 35회를 가하고, 정상 3셀은 본 타격 180회씩 수집한다. 정상 셀은 시간 순서로 학습 120·임계값 30·평가 30창으로 나뉘, 합계 학습 360·임계값 90·평가 90창을 확보한다.

데이터 수집 및 저장 형식

수집 프로그램의 동작과 기록 주기는 2.2와 같다. 원시 CSV는 타임스탬프(ns·Unix 초), DHT22 온도·상대습도, ADXL355Z 내부 온도, g 단위 3축 가속도를 열로 갖고, 파일명의 T{x,o}-H{x,o}-M{0,500,1000}-B{0,1,2} 조건 코드와 반복 횟수 메타데이터로 조건을 식별한다. 분석 단계에서는 타격별 창마다 z축 피쳐(로그 RMS·로그 p2p·피크 주파수·로그 피크 파워·로그 총에너지·대역별 에너지 비율)를 계산해 창 평균 온습도·조건과 연결하고, 피크 주파수를 1차 고유진동수 추정 지표로 사용한다.

3.2 테스트 방법

테스트는 openLAB 공개 데이터 분석과 자체 모형 교량 실험으로 나눈다. openLAB은 전 구간이 무손상 기준이어서 임계값 초과를 모두 거짓 경보로 볼 수 있어 보정 방식별 오탐율 비교에 쓴다(과정·결과는 2.7). 자체 실험은 저비용 센서 수집·질량 변화 탐지·보정 재현을 검증한다. 수집·기록 체계는 1·2차 모두 2.2, 3.1과 같으며, 아래는 실험별 차이만 기술한다.

1차는 2.2의 센서 배치로 0·0·1·0·1·1을 수동 가진해 각 20~30초 이상, 가능하면 3회 이상 반복 기록하였다. 환경 조건은 야외 온도 신뢰도 문제로 당초 12개(질량 3×환경 4)에서 9개(질량 3×환경 3: 기준·습도 변화 중심·온습도 동시 변화)로 축소하였다. 분석 창은 4초로 고정한다. 보정·이상 탐지 모델은 정상 0·0 데이터(환경 3조건)만으로 학습하고 나머지는 평가에 쓴다. 임계값은 정상 재구성 오차의 99번째 백분위수를 세 모델(무보정·온도·온습도 보정)에 동일 적용하며, 세 모델은 2.3의 PCA 재구성(선형 AE 등가)이다.

2차의 핵심 변경은 수동 가진을 솔레노이드 오토 태퍼 자동 타격으로 바꾼 것으로, 정상 기준의 산포를 줄이기 위함이다. 타격 시각과 가속도를 동일 시계로 동기화해 타격 직후 창만 분석한다.

샘플링 500 Hz·측정 레인지 ±8 g로 수집하고, 분석 창은 임팩트를 제외한 $t_0+0.05$ s부터 2.0 s(1,000샘플)로 전 조건 고정한다. 스펙트럼은 Welch로 추정하고, 분석 대역은 실측 1차 고유진동수에 맞춰 0~40·40~80·80~125 Hz로 재설정하며, 1차의 0~12·12~25·25~50 Hz와 다르다. 피쳐는 z축 8개(로그 RMS·로그 p2p·피크 주파수·로그 피크 파워·로그 총에너지·대역 에너지 비율 3종)로, 진폭은 로그, 대역 에너지는 비율로 정규화해 타격 세기가 아니라 스펙트럼 구조 변화가 드러나게 했다.

학습·임계값·평가는 데이터 누수를 막도록 분리한다. 학습은 정상 360창, 임계값은 학습에 쓰지 않은 전용 90창의 재구성 오차 99번째 백분위수, 평가는 정상 90창·손상 840창으로 수행한다(분할은 3.1). 모델은 은닉 8-3-8 비선형 Autoencoder(tanh)로 통일하고 환경 입력만 무보정·온도·온습도로 바꿔 비교하며, 난수 시드 5개로 반복해 안정성을 확인한다. 실시간 판정 체계는 2.2와 같고, 학습·평가는 저장 CSV로 수행한다.

3.3 예상 결과

본 절에서는 구조 조건 변화(질량 분포·볼트 풀림)와 온습도 보정이 진동 피쳐 및 이상 점수에 미칠 것으로 예상되는 결과와, 그 검증 계획을 서술한다.

1차에서는 질량 증가로 고유진동수가 낮아져 무게추 조건의 피크 주파수 하락이, 비대칭 1-0에서는 주파수 이동보다 진폭·대역 에너지 변동이 먼저 예상된다. 실내 온습도 변화 폭은 openLAB의 계절 변화보다 작아 보정 효과도 작을 수 있어, 절대 크기보다 개선 경향(거짓 경보 감소)의 재현 여부를 기준으로 삼는다.

2차에서는 부가 질량이 데크 질량(약 243 g)보다 크지만, 무게추가 지지점 부근에 놓여 1차 모드에 실리는 유효 질량은 겉보기보다 작다. 따라서 1차 고유진동수는 뚜렷이 하락하되 질량 비율만큼 크게 떨어지지 않을 것으로 보며, 실제 크기는 조립 후 실측으로 확정한다. 관찰의 초점은 검출 여부보다 서열이다. 같은 손상 유형 안에서 조건이 심해질수록(M0→M500→M1000, B0→B1→B2) 이상 점수와 배율이 단조 증가하는지 확인하며, 이것이 성립해야 이상 점수를 점검 우선순위의 심각도 지표로 쓸 수 있다. 서로 다른 유형 사이의 비교는 물리적 기준이 달라 정답을 정의할 수 없으므로 검증에서 제외하고, 운용에서는 정상 대비 이탈 정도의 상대 순위로만 제시한다. 볼트 풀림은 고유진동수의 소폭 하락에 고조파와 피크 폭 확대 같은 부수 변화가 동반될 것으로 예상한다. 요컨대 질량은 주파수 이동의 크기로, 볼트는 동반 피치의 조합으로 구분된다는 가설을 검증하고, 1차에서 편차로 해석한 1-0의 역방향 이동이 자동 가진에서 사라지는지, 환경만 변한 정상 3셀에서 보정 효과가 재현되는지도 함께 확인한다.

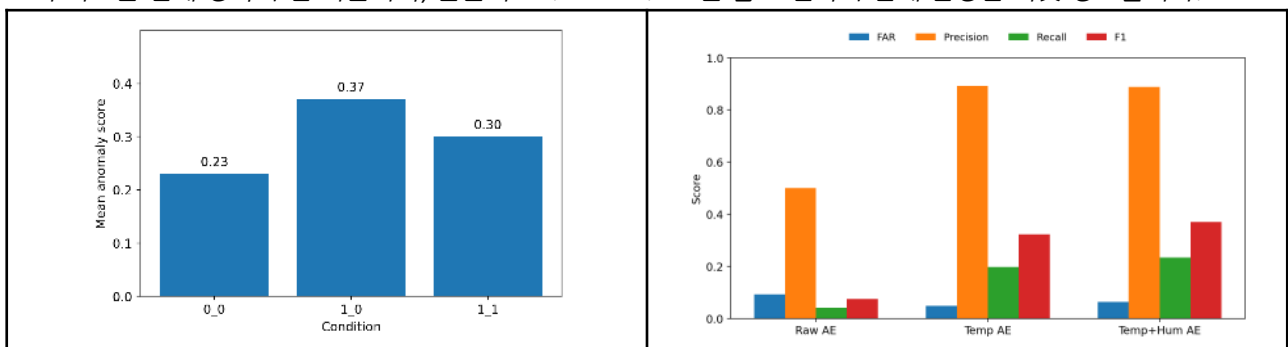
3.4 실제 결과

예비 실험에서 1차 피크가 약 22.6 Hz로 관측되어 기본 분석 대역을 0~12·12~25·25~50 Hz로 재설정하였다. 1-0의 질량 변화는 다피치 이상 점수로 감지되어 임계값 초과 비율이 29.7%까지 올랐다. 습도 변화 구간의 거짓 경보율은 무보정 17.5%에서 온습도 보정 9.1%로 줄었다. 감소 폭 8.4%p는 openLAB의 0.38%p보다 오히려 큰데, 실내에서는 습도를 크게 바꿀 수 있어 보정 전 오탐율이 높았고 그만큼 줄어들 여지도 컸기 때문이다. 1차 결과는 두 표와 그림4·그림5로 제시하며, 첫 표는 조건별 집계이고 둘째 표는 전체 평가 구간의 모델별 지표라 집계 모집단이 달라 직접 비교되지 않는다. 그림4에서는 무게추 조건(1-0·1-1)의 평균 이상 점수가 정상(0-0)보다 높아지는지를, 그림5에서는 보정 단계가 더해질수록 지표가 개선되는지를 보면 된다.

조건 ID	좌측 하중	우측 하중	평균 f	f 변화율	평균 이상 점수
0-0	0g	0g	22.6 Hz	+0.0%	0.23
1-0	100g	0g	22.8 Hz	+0.8%	0.37
1-1	100g	100g	20.6 Hz	-8.9%	0.30

모델	입력 환경 변수	False Alarm Rate	Precision	Recall	F1-score
무보정 AE	없음	0.092	0.500	0.041	0.075
온도 보정 AE	온도 T	0.048	0.892	0.197	0.323
온도·습도 보정 AE	온도 T + 상대습도 RH	0.063	0.888	0.234	0.370

표의 지표는 전체 평가 구간 기준이며, 본문의 17.5%→9.1%는 습도 변화 구간에 한정된 거짓 경보율이다.



2차 실험 결과

온습도 보정 모델에서 손상 840창이 전부 임계값을 초과해 재현율 100%였고, 정상 평가 90창 중 거짓 경보는 4건(4.4%)이었다. 이 4건은 모두 온습도 동시(To·Ho) 블록에서 발생한 동일 사건으로, 아래 표의 4/30(13.3%)은 같은 4건을 해당 블록 30창 기준으로 본 값이다. 27조건 1,380창은 폐기 없이 전부 유효하였다. 실측 정상 기준 1차 고유진동수는 101.0 Hz, 이상 판정 임계값은 재구성 오차 5.6064다. 아래 표의 평균 이상 점수는 이 임계값과 같은 척도의 절대값이고, 본문과 그림6의 배율(2.2)은 이를 임계값으로 나눈 값이다. 아래 표의 손상 조건군은 환경 3블록 합산 값이며(각 105창), 전체 27셀 분포는 그림6에 제시한다. 한편 아래 표의 피크 이동률은 3.3에서 예상한 완만한 하락보다 훨씬 크고(-64.4% 등), 손상이 더 심한데 이동은 오히려 작은 역전도 있다(M1000·B1 -19.8%). 이는 고유진동수가 실제로 그만큼 내려갔다는 뜻이 아니다. 구조물은 여러 진동 모드를 함께 갖고 피크 주파수는 그중 가장 큰 봉우리 하나만 읽는 값인데, 손상으로 모드별 세기가 달라지면 가장 큰 봉우리가 기존 101 Hz 모드에서 더 낮은 모드로 옮겨 앉는다(모드 전환). 그래서 값이 연속적으로 내려가는 대신 크게 건너뛰고, 어느 모드가 가장 커질지는 손상 크기에 단조로 비례하지 않아 역전도 자연스럽다(상세 해석은 3.5).

조건군	피크 주파수 중앙값	피크 이동률(대 101 Hz)	평균 이상 점수	임계값 초과 비율
정상 기준 (Tx·Hx·M0·B0)	101.0 Hz	+0.0%	0.03	0/30 (0%)
습도 변화 (Tx·Ho·M0·B0)	101.0 Hz	+0.0%	0.07	0/30 (0%)
온습도 동시 (To·Ho·M0·B0)	90.0 Hz	-10.9%	4.13	4/30 (13.3%)
M500·B0	36.0 Hz	-64.4%	33.2	105/105 (100%)
M1000·B0	36.0 Hz	-64.4%	128.4	105/105 (100%)
M0·B1	64.0 Hz	-36.6%	264.7	105/105 (100%)
M0·B2	40.0 Hz	-60.4%	370.3	105/105 (100%)
M500·B1	44.0 Hz	-56.4%	75.4	105/105 (100%)
M1000·B1	81.0 Hz	-19.8%	108.3	105/105 (100%)
M500·B2	66.0 Hz	-34.7%	307.1	105/105 (100%)
M1000·B2	53.0 Hz	-47.5%	323.4	105/105 (100%)

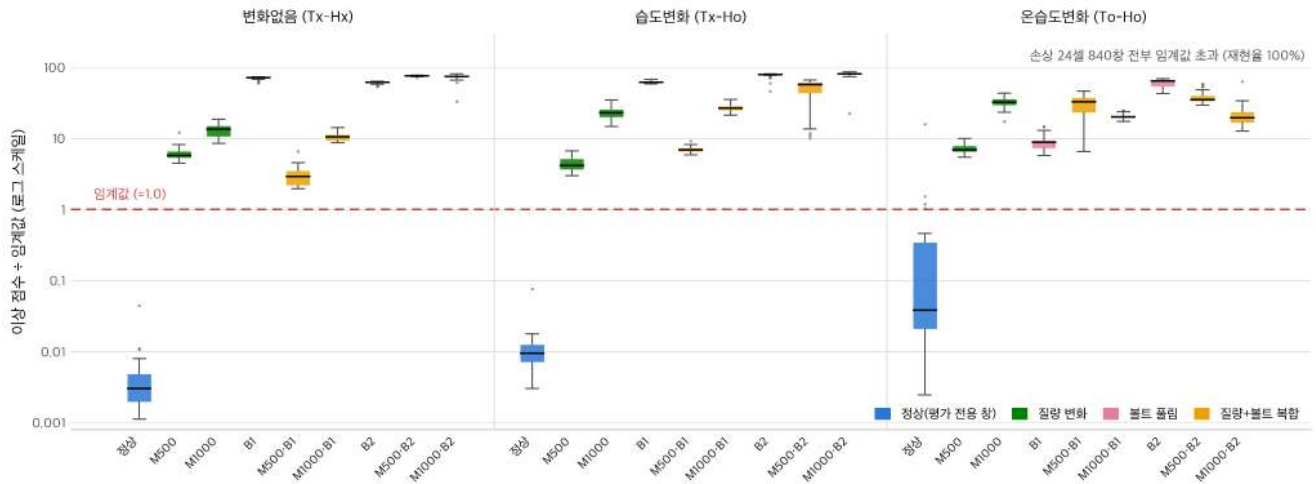
모델	False Alarm Rate	Precision	Recall	F1-score
무보정 AE (환경 입력 없음)	0.022	0.998	0.993	0.995
온도 보정 AE (T 잔차)	0.033	0.996	1.000	0.998
온도·습도 보정 AE (T·RH 잔차)	0.044	0.995	1.000	0.998

시드 5개 반복에서 보정 두 모델은 전 시드 재현율 100%를 유지했으나, 무보정 AE는 96.2~99.3%로 변동했다(해석은 3.5). 세 모델의 ROC-AUC는 0.994~0.996이다.

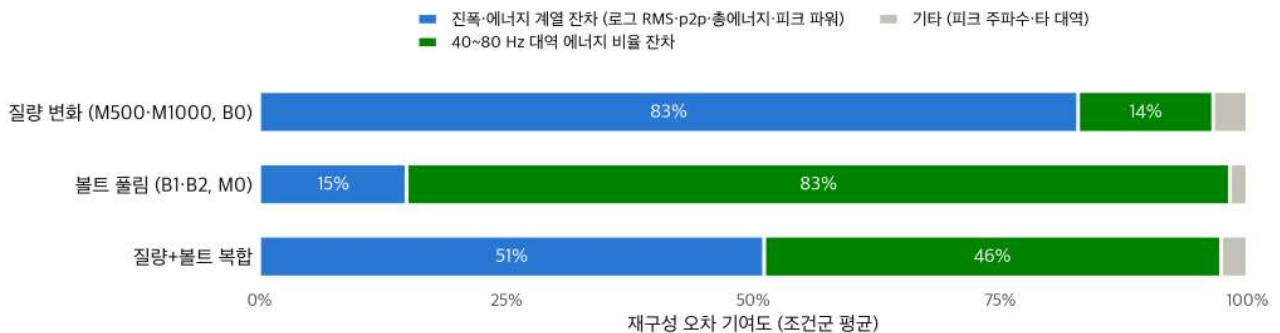
2차에서는 1차와 같은 FAR 감소가 재현되지 않았고, 보정 이득은 손상 검출 안정화로 나타났다. 환경만 변한 정상 평가 창에서 무보정 AE의 거짓 경보율부터가 0~6.7%로 낮았다. 아래 표는 두 지표를 함께 보인다(해석은 3.5).

지표	무보정 AE	온습도 보정 AE
습도 변화 블록 FAR (Tx·Ho)	0.0% (0/30)	0.0% (0/30)
온습도 동시 블록 FAR (To·Ho)	6.7% (2/30)	13.3% (4/30)
손상 재현율 (840창, 시드 5개 범위)	96.2~99.3%	100% (전 시드)

환경 거짓 경보는 두 모델이 동일하게 낮아, 실질 차이는 손상 재현율 행이다. To·Ho의 2건 대 4건 차이는 30창 기준 이항 신뢰구간이 겹쳐 같은 수준이며, 보정 오탐 4건 중 3건은 배율 1.0~1.5배의 경계 초과다.



[그림6: 2차 27조건 이상 점수 분포(임계값 대비 배율, 로그 스케일) — 손상 840창 전부 임계값 초과]



[그림7: 손상 유형별 재구성 오차 피쳐 기여도 — 질량형·볼트형·복합형 프로파일]

3.5 결과 분석

1차 결과는 예상한 물리 현상과 들어맞았다. 양측 하중 1-1에서는 평균 피크 주파수가 8.9% 낮아졌는데, 질량이 늘면 고유진동수가 낮아진다($f = (1/2\pi)\sqrt{k/m}$), 강성 k 가 같을 때 질량 m 이 커지면 f 하락)는 예상 그대로다. 주요 기여 대역도 0~12 Hz에서 1차 공진이 속한 12~25 Hz로 옮겨 가 같은 해석을 뒷받침한다. 한쪽 하중 1-0은 피크가 0.8% 올랐지만 수동 타격의 편차 범위 안이라 의미 있는 변화가 아니다. 대신 진폭·대역 피쳐가 반응해 임계값 초과 비율이 4.5%(정상 0-0 창 기준)에서 29.7%(1-0)로 뛰었다. 작은 비대칭 질량 변화는 주파수 하나가 아니라 여러 피쳐를 종합한 이상 점수로 잡아야 한다는 설계 가정이 확인된 것이다. 남은 문제는 폼보드의 감쇠로 공진 피크가 흐릿해 작은 주파수 변화가 피크 폭에 묻히고, 수동 타격은 세기·시점이 매번 달라 손상이 없어도 정상 측정값의 산포 자체가 커진다는 점이었다. 정상 산포 밖을 이상으로 잡는 비지도 탐지에서 정상 산포가 넓다는 것은 그만큼 작은 손상을 놓친다는 뜻이라, 이 둘이 2차의 개선 목표가 되었다.

2차 결과가 보여주는 것은 네 가지다.

첫째, 가진을 기계로 표준화하자 정상 기준이 고정되었다. 오토 태퍼 도입 후 상온 정상 평가창 60개의 피크가 101.0 Hz 하나의 값에 표준편차 0.0 Hz로 수렴했다. 1차에서 문제가 되었던 수동 타격의 세기·시점 편차가 사라졌다는 뜻이고, 1차에서 편차 탓으로 해석했던 0.8%의 역방향 피크 이동도 다시 나타나지 않았다.

둘째, 27조건이 사실상 완전히 구분되었다. 수치는 3.4의 표와 같고, 핵심은 정상과 손상 사이의 간격이다. 배율 중앙값이 정상 창은 0.003~0.038배, 손상 창은 5.8~73.7배로 백 배 넘게 벌어졌다(그림6). 다만 꼬리까지 갈라진 것은 아니어서, 가열 블록의 정상 창 하나가 15.9배까지 튀었고 가장 약한 손상 창은 2.0배에 그쳤다. 그래서 실제 운용에서는 창 하나가 아니라 연속된 창의 초과 비율로 판단한다. 심각도 서열은 질량에서 깨끗했다. 배율이 M500의 5.8배에서 M1000의 22.6배로 커지는 단조 증가가 세 블록 모두에서 나타났다. 볼트는 한 개만 풀어도 점수가 수십 배로 포화되어 서열이 저심각도 구간에서만 유효했고, 변화없음 블록에서는 B1의 71.5배가 B2의 61.5배를 앞서는 역전도 있었다.

셋째, 재구성 오차의 기여도 구성이 손상의 종류를 가리킨다(그림7). 볼트가 풀린 창에서는 40~80 Hz 대역 비율 잔차가 재구성 오차의 76~90%를, 질량을 올린 창에서는 로그 RMS 같은 진폭·에너지 계열이 73~80%를 차지했다. 반대 방향으로 읽으면 운용 지침이 된다. 알람의 기여도에서 40~80 Hz 대역 비율이 지배적이면 볼트 체결부 같은 경계 조건의 손상을, 진폭·에너지 계열이 지배적이면 하중(질량) 변화를 먼저 의심하고 점검을 시작하면 된다. 질량+볼트 복합 창에서 두 계열의 기여가 51%·46%로 나란히 나타난 것도 이 내용을 뒷받침한다. 위치와 종류를 확정하는 진단은 아니지만 후속 정밀 점검의 출발점을 정해 준다. 한편 피크 주파수는 질량·볼트 모두 모드 전환(3.4) 탓에 예상보다 훨씬 크게 흔들렸는데(볼트 B1 -36.6%·B2 -60.4%), 피크 하나에 기대지 않고 여러 피처를 종합해 탐지한다는 1차의 결론이 2차에서도 유효했음을 보여 준다.

넷째, 온습도 보정의 역할이 1차와 달라졌다. 계단식 가열과 연속 가습 덕에 정상 학습 데이터가 온도 22.4~39.9 °C, 습도 48.9~79.0%를 미리 포괄했고, 보정 없이도 환경 변화가 거짓 경보로 이어지는 일이 드물어 1차와 같은 감소 폭이 나타날 자리가 없었다. 대신 보정의 이득은 검출 안정화로 나타났다. 시드 5개로 학습을 반복하면 무보정 모델의 재현율은 96.2~99.3%로 흔들렸지만 보정 모델은 5개 전부에서 100%였다.

성능 개선의 가장 큰 몫은 피처 정규화다. 조건 사이 진폭이 약 40배까지 차이 나는 탓에 1차의 원시 피처로는 표준화가 진폭에 눌러 재현율이 21.3%(2차 데이터에 원시 피처를 그대로 적용한 재분석 기준), 질량 창에서는 0%에 그쳤지만, 로그·비율 정규화로 바꾸자 같은 데이터에서 100%가 되었다.

남은 거짓 경보 4건은 모두 가열이 가장 뜨거웠던 39.9 °C 부근 창이다. 공기 온습도만으로 구조물 온도를 근사하는 한계이며, 개선은 4.2에서 다룬다. 1차·2차 실험의 개선 사항을 요약하면 아래 표와 같다.

항목	1차 실험	2차 실험
모형·가진	폼보드 모형(55 cm)·수동 타격	알루미늄 다경간 모형(60 cm)·솔레노이드 오토 태퍼(40 ms·5 s 주기)
조건·분석 창	9조건·4초 창(200 Hz)	27조건 1,380창·2.0초 창(500 Hz)
정상 피크 재현성	22.6 Hz, 수동 편차로 창별 변동	101.0 Hz, 상온 평가창 60개 표준편차 0.0 Hz
손상 검출	창 단위 초과 비율 18.8~29.7%(1·1·1·0)	창 단위 재현율 100% (840/840)
거짓 경보	습도 구간 9.1% (보정 후)	평가 창 전체 4.4% (4/90)
모델·피처	PCA 재구성(선형 등가)·원시 스케일 피처	비선형 AE(8·3·8)·로그·비율 정규화 피처

4.1 결론

본 시스템은 저비용 상용 부품(ADXL355Z·DHT22·Raspberry Pi)만으로 실험실 모형의 27조건 매트릭스에서 정밀 점검 이전 단계의 1차 스크리닝 목표 성능을 입증했다. 온습도 보정 모델은 손상 840창을 전부 잡아내고 정상 평가 90창 중 4건만 잘못 올려, 창 단위 재현율 100%와 거짓 경보율 4.4%, F1 0.998, ROC-AUC 0.996을 기록했다. 1차 실험에서는 Precision 0.888에 Recall 0.234로 경보가 올리면 대체로 맞지만 손상을 자주 놓치는 시스템이었다면, 2차 실험에서는 가진 표준화와 피쳐 정규화 재설계를 거치며 놓침 없이 정확히 올리는 시스템이 되었다.

온습도 보정의 가치는 세 갈래에서 서로 다른 모습으로 확인되었다. 학습이 환경 변동을 충분히 보지 못한 조건에서는 거짓 경보를 직접 줄였고(1차 17.5%→9.1%, openLAB 1.90%→1.52%), 환경 커버리지를 확보한 2차에서는 난수 시드와 무관하게 재현율 100%를 유지시키는 검출 안정화로 나타났다. 여기에 재구성 오차의 기여도 분석이 알람의 질을 바꾼다. 이상 신호만 주는 시스템에서는 점검자가 무엇을 살펴야 할지 알 수 없지만, 본 시스템은 어느 피쳐가 오차를 키웠는지를 함께 제시한다. 2차에서 볼트 풀림은 40~80 Hz 대역 비율 잔차가 기여도의 76~90%를, 질량 변화는 진폭·에너지 계열 잔차가 73~80%를 차지해, 알람만 보고도 경계부 이상인지 하중 변화인지 방향을 좁힐 수 있었다. 손상의 위치나 종류를 확정하는 것은 아니지만, 후속 정밀 점검이 어디서 시작할지 정하는 실질적 단서가 된다. 유형 내 심각도 서열도 질량형에서는 전 블록 성립해 점검 우선순위 지표로 쓸 근거를 얻었고, 볼트형에서는 점수가 포화되기 전인 B0→B1의 저심각도 구간에 한해 성립했다.

모델 측면에서는 로그·비율 정규화 피쳐를 전제로 한 은닉 8-3-8 비선형 Autoencoder가 1차의 선형 PCA 구현을 대체했는데, 성능을 끌어올린 지배 요인은 모델 용량보다 피쳐 표현이었다. 다만 이 모든 수치는 단기·실내·모형 실험의 결과다. 실교량의 교통 하중과 계절 사이클, 장기 드리프트는 아직 겪지 않았으므로, 실용화 판단은 4.2의 실증을 통과한 뒤의 일이다.

4.2 추후 개선사항

추후 개선은 실교량 실증 하나에 집중한다. 모형에서 확인할 것은 2차에서 확인했으므로, 남은 질문은 실제 운영 환경에서도 같은 성능이 나오는가이다. 센서 자체의 계측 성능은 같은 ADXL355 계열로 실구조물 모달 계측을 검증한 선행연구[1]에 기대고, 대신 후보 교량 여러 곳에서 단기 측정으로 상시 진동의 신호 대 잡음비(SNR)를 비교해 공진 피크가 가장 선명한 곳을 대상으로 고른다. 대상은 최근 정밀점검을 통과한 보행자 육교(제3종 규모)로 한정해 측정 시작 시점의 정상 가정을 점검 기록으로 뒷받침하고, 주경간 중앙부 하면에 단일 노드를 방수 외함·전원·통신과 함께 설치하며, 가지는 보행과 바람이 만드는 상시 진동으로 전환한다.

다음은 실환경 보정 효과의 정량화다. 설치 후 첫 1주일의 정상 데이터로 회귀·Autoencoder·임계값을 현장 학습한 뒤, 2~3주의 무손상 상시 감시로 무보정 모델과 보정 모델의 거짓 경보율을 나란히 비교해, 실교량의 일교차·기상 변동에서 온습도 보정이 거짓 경보를 얼마나 줄이는지를 자체 시스템 데이터로 산출한다. 창 단위 거짓 경보율과 연속 창 오경보 횟수의 상한은 감시 시작 전에 선언해 두고, 월 단위 재학습·임계값 갱신 절차와 센서 재부착 시의 기준선 드리프트 측정도 이 기간에 확립하며, 구조물 온도 근사의 한계는 데크 표면 온도 센서를 추가해 보완한다. 사계절 검증은 노드 상설 운영으로 이어갈 장기 과제다.

마지막으로 검출력을 확인한다. 실교량에는 손상을 가할 수 없으므로, 관리 주체 협조 하에 알려진 질량을 지정 위치에 두는 가역적 통제 교란을 감시 중간과 종료 시점에 삽입한다. 교란 크기는 문헌상 실제 손상이 만드는 고유진동수 변화(수 %)에 맞춰 사전 계산하고, 하중 2~3수준으로 2차의 심각도 서열이 재현되는지 확인한다. 이 세 단계를 통과하면 정기 점검과의 대조, 다중 노드 확장과 손상 위치 추정 검토를 거쳐 중소형 교량 상시 스크리닝 수단으로 나아갈 검증 기반이 갖춰진다.

참고문헌

- [1] Caballero-Russi, D., Ortiz, A. R., Guzmán, A., & Canchila, C., “Design and Validation of a Low-Cost Structural Health Monitoring System for Dynamic Characterization of Structures,” Applied Sciences, 12(6), 2807, 2022.

- [2] Peeters, B., & De Roeck, G., “One-year monitoring of the Z24-Bridge: environmental effects versus damage events,” Earthquake Engineering & Structural Dynamics, 30(2), 149-171, 2001.

- [3] Z24 Bridge Benchmark documentation, KU Leuven, Leuven, Belgium.

- [4] 한국표준과학연구원(KRISS), BOCDA(Brillouin Optical Correlation Domain Analysis) 기반 분포형 광섬유 센서 기술.

- [5] Giglioni, V. et al., “Deep Autoencoders for Unsupervised Damage Detection with Application to the Z24 Benchmark Bridge,” IMAC, Springer, 2022.

- [6] Pakzad, S. S., & Masoodi, A. R., “Early damage detection in bridges using a variational autoencoder-based hybrid unsupervised learning framework,” Scientific Reports, 16, 1389, 2026. DOI: 10.1038/s41598-025-31115-w

- [7] Aosong Electronics, DHT22(AM2302) 온습도 센서 Datasheet.

- [8] Jansen, A., Richter, B., Röder, R., Herbers, M., & Marx, S., "Monitoring Data of the openLAB Research Bridge (2024-02-01 to 2024-10-31) and building information," OPARA - Open Access Repository and Archive, TU Dresden, 2025. DOI: 10.25532/OPARA-660.

자료 목록

번호	자료형태	본문 표기	자료 설명(키워드)	비고(출처 등)
1	사진	자료1	정자교 붕괴 현장 사진	안전저널 보도(2023.4.7.), https://www.anjunj.com/news/articleView.html?idxno=36589
2	사진	자료2	Fern Hollow Bridge 붕괴 사진	NTSB 촬영(미 연방정부 저작물·퍼블릭 도메인), Wikimedia Commons 게재
3	차트	자료3	전국 교량 중 제3종 교량의 비율	국토안전관리원 2025년 시설물 종별 현황 자료 기반 직접 작성
4	표	자료4	시설물 등급별 점검 주기 및 대상 표	「시설물의 안전 및 유지관리에 관한 특별법 시행령」 별표 3(2025.12.4 개정) 기반 직접 작성
5	도식·그래프	그림1~그림7	배치 개념도, Autoencoder 구조, 시스템 순서도, 실험 결과 그래프	직접 제작
6	사진	사진 일체	모형 교량, 완성 모듈, 2차 모형 제작 과정 사진	직접 촬영

7	공개 데이터	데이터셋	openLAB(TU Dresden, OPARA — 2.7 분석에 사용), Z24 Bridge Benchmark(KU Leuven — 선행 연구 비교 참조)	공개 데이터, 참고문헌 3·8
8	오픈소스 SW	-	Python, NumPy, pandas, scikit-learn, Matplotlib (수집·분석·시각화)	각 공식 배포본(PSF·BSD 라이선스)

소스코드

Sentry.py

```

from __future__ import annotations

import argparse
import io
import json
import math
import socket
import subprocess
import sys
import threading
import time
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from flask import Flask, jsonify
from scipy.signal import welch
from sklearn.linear_model import LinearRegression
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler

HTML = r'''<!doctype html>
<html lang="ko">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>SENTRY</title>
<style>
body{margin:0;background:#10141b;color:#eef2f7;font-family:Arial,sans-serif}.wrap{
max-width:1200px;margin:auto;padding:20px}.top{display:flex;justify-content:space-
between;align-items:center;margin-bottom:16px}.title{font-size:28px;font-weight:70
0}.grid{display:grid;grid-template-columns:repeat(6,1fr);gap:12px}.card{background
:#1a202b;border-radius:14px;padding:16px;box-shadow:0 6px 18px
#0004}.wide{grid-column:span 3}.full{grid-column:span
6}.label{font-size:13px;color:#9ca8b8}.value{font-size:28px;font-weight:700;margin
-top:6px}.status{padding:10px
18px;border-radius:999px;font-weight:700}.normal{background:#176b45}.warning{backg
round:#916b12}.anomaly{background:#9d2c2c}.waiting{background:#465164}canvas{width
:100%;height:250px;background:#111722;border-radius:10px}.bands{display:flex;heigh
t:26px;border-radius:8px;overflow:hidden;margin-top:12px}.band{display:flex;align-
items:center;justify-content:center;min-width:0;font-size:12px}.b1{background:#3f8
2d3}.b2{background:#c98b2e}.b3{background:#bd4f59}.small{font-size:12px;color:#9ca
8b8;margin-top:8px}@media(max-width:800px){.grid{grid-template-columns:repeat(2,1f
r)}.wide,.full{grid-column:span 2}}
</style>
</head>

```

```

<body>
<div class="wrap">
<div class="top"><div><div class="title">SENTRY 실시간 대시보드</div><div
id="connection" class="small">서버 연결 확인 중</div></div><div id="status"
class="status waiting">대기 중</div></div>
<div class="grid">
<div class="card"><div class="label">이상 점수 / 임계값</div><div id="score"
class="value">-</div></div>
<div class="card"><div class="label">피크 주파수</div><div id="peak"
class="value">-</div></div>
<div class="card"><div class="label">RMS</div><div id="rms"
class="value">-</div></div>
<div class="card"><div class="label">온도</div><div id="temp"
class="value">-</div></div>
<div class="card"><div class="label">습도</div><div id="hum"
class="value">-</div></div>
<div class="card"><div class="label">누적 이상 후보</div><div id="detection"
class="value">-</div></div>
<div class="card wide"><div class="label">최근 z축 파형</div><canvas
id="wave"></canvas></div>
<div class="card wide"><div class="label">최근 타격 주파수 스펙트럼</div><canvas
id="spec"></canvas></div>
<div class="card full"><div class="label">대역별 재구성 오차 기여도</div><div
class="bands"><div id="b1" class="band b1"></div><div id="b2" class="band
b2"></div><div id="b3" class="band b3"></div></div><div id="info"
class="small">-</div></div>
</div>
</div>
<script>
function line(canvas,x,y){const
c=document.getElementById(canvas),d=devicePixelRatio||1,w=c.clientWidth,h=c.clientHeight;c.width=w*d;c.height=h*d;const
g=c.getContext('2d');g.scale(d,d);g.clearRect(0,0,w,h);g.strokeStyle='#2c3544';g.beginPath();for(let i=1;i<5;i++){let
yy=h*i/5;g.moveTo(0,yy);g.lineTo(w,yy)}g.stroke();if(!y||y.length<2)return;let
ymin=Math.min(...y),ymax=Math.max(...y);if(ymax===ymin){ymax+=1;ymin-=1}g.strokeStyle='#73b7ff';g.lineWidth=2;g.beginPath();for(let i=0;i<y.length;i++){let
px=i/(y.length-1)*w,py=h-(y[i]-ymin)/(ymax-ymin)*h;if(i===0)g.moveTo(px,py);else
g.lineTo(px,py)}g.stroke()}
function ratioText(v){if(v==null)return '-';if(v>=1000)return
v.toExponential(2)+'x';return v.toFixed(2)+'x'}
async function update(){const conn=document.getElementById('connection');try{const
r=await fetch('/api',{cache:'no-store'});if(!r.ok)throw new Error('api');const
s=await r.json();const
st=document.getElementById('status');st.textContent=s.status_text;st.className='st
atus
'+s.status_class;document.getElementById('score').textContent=ratioText(s.score_ra
tio);document.getElementById('peak').textContent=s.peak_hz==null?'-':s.peak_hz.toF
ixed(2)+'
Hz';document.getElementById('rms').textContent=s.rms==null?'-':s.rms.toFixed(4)+'
g';document.getElementById('temp').textContent=s.temp_c==null?'-':s.temp_c.toFixed
(1)+'
°C';document.getElementById('hum').textContent=s.humidity==null?'-':s.humidity.toF
ixed(1)+'
%';document.getElementById('detection').textContent=s.evaluated_count?String(s.ano
maly_count)+' /
'+String(s.evaluated_count):'-';line('wave',s.wave_x,s.wave_y);line('spec',s.freq_
x,s.freq_y);const b=s.bands||[0,0,0],ids=['b1','b2','b3'];for(let
i=0;i<3;i++){const
e=document.getElementById(ids[i]);e.style.width=(b[i]*100)+'%';e.textContent=b[i]>
0.08?Math.round(b[i]*100)+'%':''}document.getElementById('info').textContent=s.inf
o||'-';if(s.updated_at==null){conn.textContent='서버 연결됨 · 데이터 대기'}else{const
age=Math.max(0,Date.now()/1000-s.updated_at);conn.textContent=age<3?'서버 연결됨 ·
실시간 수집 중': '서버 연결됨 · 마지막 데이터 '+age.toFixed(1)+'초

```

```

전'}}catch(e){conn.textContent='서버 연결 끊김';const
st=document.getElementById('status');st.textContent='연결
끊김';st.className='status waiting'}}
setInterval(update,700);update();
</script>
</body></html>'

```

```

RAW_FEATURES = ["rms", "std", "p2p", "peak_hz", "peak_power", "energy_1",
"energy_2", "energy_3"]
FEATURES = ["log_rms", "log_p2p", "peak_hz", "log_peak_power", "log_total_energy",
"band_frac_1", "band_frac_2", "band_frac_3"]
FILTER_LPF = {"0x03": 125.0, "0x04": 62.5, "0x05": 31.25}
EPS = 1e-12

```

```

def prepare_features(raw: pd.DataFrame | np.ndarray) -> np.ndarray:
    if isinstance(raw, pd.DataFrame):
        x = raw[RAW_FEATURES].to_numpy(float)
    else:
        x = np.asarray(raw, dtype=float)
        if x.ndim == 1:
            x = x.reshape(1, -1)
    total = x[:, 5] + x[:, 6] + x[:, 7] + EPS
    return np.column_stack([
        np.log10(np.maximum(x[:, 0], EPS)),
        np.log10(np.maximum(x[:, 2], EPS)),
        x[:, 3],
        np.log10(np.maximum(x[:, 4], EPS)),
        np.log10(total),
        x[:, 5] / total,
        x[:, 6] / total,
        x[:, 7] / total,
    ])

```

```

def model_features(model: dict, raw: pd.DataFrame | np.ndarray) -> np.ndarray:
    if model.get("feature_version") == "log_normalized_v1":
        return prepare_features(raw)
    if isinstance(raw, pd.DataFrame):
        return raw[RAW_FEATURES].to_numpy(float)
    x = np.asarray(raw, dtype=float)
    return x.reshape(1, -1) if x.ndim == 1 else x

```

```

def companion_paths(bridge: Path) -> tuple[Path, Path]:
    name = bridge.name
    if not name.endswith("_bridge_log.csv"):
        raise ValueError("bridge CSV 이름은 *_bridge_log.csv 형식이어야 합니다.")
    base = name[:-len("_bridge_log.csv")]
    return bridge.with_name(base + "_tap_events.csv"), bridge.with_name(base +
"_metadata.json")

```

```

def metadata_for(bridge: Path) -> dict:
    _, meta = companion_paths(bridge)
    if meta.exists():
        return json.loads(meta.read_text(encoding="utf-8"))
    return {"sample_rate_hz": 500, "range_g": 8, "filter_register_hex": "0x03"}

```

```

def limits(meta: dict) -> tuple[float, float, list[float]]:
    fs = float(meta.get("sample_rate_hz", 500))
    lpf = FILTER_LPF.get(str(meta.get("filter_register_hex", "0x03")).lower(),
min(fs / 2, 125.0))

```

```

top = min(fs / 2, lpf)
if top >= 120:
    bands = [0.0, 40.0, 80.0, top]
else:
    bands = [0.0, top / 3, top * 2 / 3, top]
return fs, top, bands

def read_taps(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path)
    if "pulse_start_perf_ns" in df.columns:
        df["tap_ns"] = pd.to_numeric(df["pulse_start_perf_ns"], errors="coerce")
    elif "t_perf_ns" in df.columns:
        df["tap_ns"] = pd.to_numeric(df["t_perf_ns"], errors="coerce")
    else:
        raise ValueError("tap CSV에 pulse_start_perf_ns 또는 t_perf_ns가 필요합니다.")
    return df.dropna(subset=["tap_ns"]).reset_index(drop=True)

def extract_one(seg: pd.DataFrame, fs: float, top: float, bands: list[float]) ->
tuple[np.ndarray, float, float, np.ndarray, np.ndarray]:
    z = pd.to_numeric(seg["az_g"], errors="coerce").to_numpy(float)
    z = z[np.isfinite(z)]
    if len(z) < max(64, int(fs * 0.5)):
        raise ValueError("분석 구간의 유효 샘플이 부족합니다.")
    z = z - np.mean(z)
    rms = float(np.sqrt(np.mean(z * z)))
    std = float(np.std(z))
    p2p = float(np.ptp(z))
    nperseg = min(len(z), max(256, int(fs)))
    freq, power = welch(z, fs=fs, nperseg=nperseg)
    valid = (freq >= 1.0) & (freq <= top)
    if not np.any(valid):
        raise ValueError("유효 주파수 대역이 없습니다.")
    vf, vp = freq[valid], power[valid]
    idx = int(np.argmax(vp))
    peak_hz = float(vf[idx])
    peak_power = float(vp[idx])
    energies = []
    for lo, hi in zip(bands[:-1], bands[1:]):
        m = (freq >= lo) & (freq < hi)
        energies.append(float(np.trapezoid(power[m], freq[m]))) if np.any(m) else
0.0)
    vector = np.array([rms, std, p2p, peak_hz, peak_power, *energies], dtype=float)
    return vector, peak_hz, rms, vf, vp

def extract_events(bridge: Path, delay: float, window: float) ->
tuple[pd.DataFrame, np.ndarray, dict]:
    tap_path, _ = companion_paths(bridge)
    meta = metadata_for(bridge)
    fs, top, bands = limits(meta)
    data = pd.read_csv(bridge)
    taps = read_taps(tap_path)
    t = pd.to_numeric(data["t_perf_ns"], errors="coerce").to_numpy(float)
    rows, env = [], []
    for _, tap in taps.iterrows():
        start = float(tap["tap_ns"]) + delay * 1e9
        end = start + window * 1e9
        a, b = np.searchsorted(t, [start, end])
        if b - a < max(64, int(fs * window * 0.7)):
            continue
        seg = data.iloc[a:b]
        try:

```

```

        vector, _, _, _ = extract_one(seg, fs, top, bands)
    except ValueError:
        continue
    temp = float(pd.to_numeric(seg.get("temp_c"), errors="coerce").mean()) if
"temp_c" in seg else math.nan
    hum = float(pd.to_numeric(seg.get("humidity"), errors="coerce").mean()) if
"humidity" in seg else math.nan
    rows.append(vector)
    env.append([temp, hum])
    if not rows:
        raise RuntimeError("유효한 타격 구간을 찾지 못했습니다.")
    return pd.DataFrame(rows, columns=RAW_FEATURES), np.asarray(env, dtype=float),
{"fs": fs, "top": top, "bands": bands}

def fill_env(env: np.ndarray, medians: np.ndarray | None = None) ->
tuple[np.ndarray, np.ndarray]:
    x = env.copy()
    if medians is None:
        medians = np.nanmedian(x, axis=0)
        medians = np.where(np.isfinite(medians), medians, 0.0)
    for j in range(x.shape[1]):
        x[~np.isfinite(x[:, j]), j] = medians[j]
    return x, medians

def train(args: argparse.Namespace) -> None:
    bridges = [Path(value) for value in args.bridge]
    parts = []
    file_peak_medians = []
    config = None
    requested = args.train_count + args.threshold_count + args.test_count
    for bridge in bridges:
        feat_part, env_part, part_config = extract_events(bridge, args.delay,
args.window)
        if config is None:
            config = part_config
        elif part_config != config:
            raise RuntimeError("모든 정상 파일은 같은 샘플링 주파수와 분석 대역을 사용해야
합니다.")
        if len(feat_part) < requested:
            raise RuntimeError(f"{bridge.name}: 유효 정상 타격이 {requested}회보다
적습니다.")
        parts.append((bridge, feat_part.reset_index(drop=True), env_part))
        file_peak_medians.append((bridge.name,
float(feat_part["peak_hz"].median())))

    train_raw = pd.concat([part.iloc[:args.train_count] for _, part, _ in parts],
ignore_index=True)
    threshold_raw = pd.concat([part.iloc[args.train_count:args.train_count +
args.threshold_count] for _, part, _ in parts], ignore_index=True)
    test_raw = pd.concat([part.iloc[args.train_count +
args.threshold_count:requested] for _, part, _ in parts], ignore_index=True)
    train_env_raw = np.vstack([env[:args.train_count] for _, _, env in parts])
    threshold_env_raw = np.vstack([env[args.train_count:args.train_count +
args.threshold_count] for _, _, env in parts])
    test_env_raw = np.vstack([env[args.train_count +
args.threshold_count:requested] for _, _, env in parts])

    train_feat = prepare_features(train_raw)
    threshold_feat = prepare_features(threshold_raw)
    test_feat = prepare_features(test_raw)
    train_env, medians = fill_env(train_env_raw)
    threshold_env, _ = fill_env(threshold_env_raw, medians)

```

```

test_env, _ = fill_env(test_env_raw, medians)
spread = np.nanstd(train_env, axis=0)
use_env = bool(spread[0] >= 0.2 or spread[1] >= 1.0)
env_model = None
if use_env:
    env_model = LinearRegression().fit(train_env, train_feat)
    train_feat = train_feat - env_model.predict(train_env)
    threshold_feat = threshold_feat - env_model.predict(threshold_env)
    test_feat = test_feat - env_model.predict(test_env)

scaler = StandardScaler().fit(train_feat)
x_train = scaler.transform(train_feat)
x_threshold = scaler.transform(threshold_feat)
ae = MLPRegressor(
    hidden_layer_sizes=(8, 3, 8),
    activation="tanh",
    solver="adam",
    alpha=1e-4,
    batch_size=min(32, len(x_train)),
    learning_rate_init=1e-3,
    max_iter=500,
    early_stopping=True,
    validation_fraction=0.15,
    n_iter_no_change=30,
    random_state=args.seed,
).fit(x_train, x_train)
threshold_error = np.mean((ae.predict(x_threshold) - x_threshold) ** 2, axis=1)
threshold = float(np.quantile(threshold_error, 0.99))
x_test = scaler.transform(test_feat)
test_error = np.mean((ae.predict(x_test) - x_test) ** 2, axis=1)
far = float(np.mean(test_error > threshold))
model = {
    "features": FEATURES,
    "raw_features": RAW_FEATURES,
    "feature_version": "log_normalized_v1",
    "config": config,
    "delay": args.delay,
    "window": args.window,
    "env_medians": medians,
    "env_model": env_model,
    "environment_enabled": use_env,
    "environment_variant": "temp_hum",
    "scaler": scaler,
    "autoencoder": ae,
    "threshold": threshold,
    "train_count": len(train_raw),
    "threshold_count": len(threshold_raw),
    "test_count": len(test_raw),
    "normal_test_scores": test_error.tolist(),
    "baseline_files": [str(path) for path in bridges],
}
joblib.dump(model, args.model)
print(f"모델 저장: {args.model}")
print(f"정상 파일: {len(bridges)}개")
print(f"타격 분할: 학습 {len(train_raw)}, 임계값 {len(threshold_raw)}, 정상 테스트 {len(test_raw)}")
print(f"환경 보정: {'사용' if use_env else '미사용'}")
print(f"정상 테스트 FAR: {far:.4f}")
if len(file_peak_medians) > 1:
    peak_values = np.asarray([value for _, value in file_peak_medians],
dtype=float)
    peak_gap = float(np.nanmax(peak_values) - np.nanmin(peak_values))
    peak_reference = float(np.nanmedian(peak_values))
    if peak_gap > max(5.0, peak_reference * 0.10):

```

```

        print("경고: 정상 파일 사이의 대표 피크 주파수 차이가 큼니다. 센서, 지지점, 태퍼,
구조 상태가 같은지 확인해야 합니다.")
        for name, value in file_peak_medians:
            print(f"    {name}: {value:.2f} Hz")

def score_features(model: dict, feat: pd.DataFrame, env: np.ndarray) ->
np.ndarray:
    x = model_features(model, feat)
    env_filled, _ = fill_env(env, np.asarray(model["env_medians"]))
    if model["env_model"] is not None:
        x = x - model["env_model"].predict(env_filled)
    scaled = model["scaler"].transform(x)
    recon = model["autoencoder"].predict(scaled)
    return np.mean((recon - scaled) ** 2, axis=1)

def wilson_interval(successes: int, total: int, z: float = 1.96) -> tuple[float,
float]:
    if total <= 0:
        return math.nan, math.nan
    p = successes / total
    denominator = 1.0 + z * z / total
    center = (p + z * z / (2.0 * total)) / denominator
    half = z * math.sqrt(p * (1.0 - p) / total + z * z / (4.0 * total * total)) /
denominator
    return max(0.0, center - half), min(1.0, center + half)

def evaluation_row(model: dict, bridge: Path, label: str) -> tuple[dict,
np.ndarray]:
    feat, env, _ = extract_events(bridge, model["delay"], model["window"])
    scores = score_features(model, feat, env)
    threshold = float(model["threshold"])
    ratios = scores / threshold
    detected = int(np.sum(scores > threshold))
    low, high = wilson_interval(detected, len(scores))
    row = {
        "condition": bridge.name.replace("_bridge_log.csv", ""),
        "label": label,
        "n": len(scores),
        "anomaly_count": detected,
        "detection_rate": detected / len(scores),
        "ci95_low": low,
        "ci95_high": high,
        "score_ratio_median": float(np.median(ratios)),
        "score_ratio_q1": float(np.quantile(ratios, 0.25)),
        "score_ratio_q3": float(np.quantile(ratios, 0.75)),
        "peak_hz_mean": float(feat["peak_hz"].mean()),
        "peak_hz_std": float(feat["peak_hz"].std(ddof=0)),
        "rms_mean": float(feat["rms"].mean()),
        "rms_std": float(feat["rms"].std(ddof=0)),
    }
    return row, scores

def evaluate(args: argparse.Namespace) -> None:
    model = joblib.load(args.model)
    rows = []
    normal_scores = []
    anomaly_scores = []
    stored_normal = np.asarray(model.get("normal_test_scores", []), dtype=float)
    if len(stored_normal):
        threshold = float(model["threshold"])

```

```

        detected = int(np.sum(stored_normal > threshold))
        low, high = wilson_interval(detected, len(stored_normal))
        ratios = stored_normal / threshold
        rows.append({
            "condition": "held_out_normal", "label": "normal", "n":
len(stored_normal),
            "anomaly_count": detected, "detection_rate": detected /
len(stored_normal),
            "ci95_low": low, "ci95_high": high,
            "score_ratio_median": float(np.median(ratios)),
            "score_ratio_q1": float(np.quantile(ratios, 0.25)),
            "score_ratio_q3": float(np.quantile(ratios, 0.75)),
            "peak_hz_mean": math.nan, "peak_hz_std": math.nan,
            "rms_mean": math.nan, "rms_std": math.nan,
        })
        normal_scores.extend(stored_normal.tolist())
    for value in args.normal:
        row, scores = evaluation_row(model, Path(value), "normal")
        rows.append(row)
        normal_scores.extend(scores.tolist())
    for value in args.anomaly:
        row, scores = evaluation_row(model, Path(value), "anomaly")
        rows.append(row)
        anomaly_scores.extend(scores.tolist())
    result = pd.DataFrame(rows)
    result.to_csv(args.output, index=False)
    if len(result):
        print(result.to_string(index=False, float_format=lambda value:
f"{value:.4f}"))
    print(f"결과 저장: {args.output}")
    if normal_scores and anomaly_scores:
        threshold = float(model["threshold"])
        normal_pred = np.asarray(normal_scores) > threshold
        anomaly_pred = np.asarray(anomaly_scores) > threshold
        tn, fp = int(np.sum(~normal_pred)), int(np.sum(normal_pred))
        tp, fn = int(np.sum(anomaly_pred)), int(np.sum(~anomaly_pred))
        precision = tp / (tp + fp) if tp + fp else math.nan
        recall = tp / (tp + fn) if tp + fn else math.nan
        f1 = 2 * precision * recall / (precision + recall) if precision + recall
else math.nan
        far = fp / (fp + tn) if fp + tn else math.nan
        print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")
        print(f"FAR={far:.4f}, Precision={precision:.4f}, Recall={recall:.4f},
F1={f1:.4f}")

def tail_dataframe(path: Path, rows: int) -> pd.DataFrame:
    with path.open("rb") as f:
        header = f.readline().decode("utf-8").strip()
        f.seek(0, 2)
        pos = f.tell()
        blocks = []
        count = 0
        while pos > 0 and count <= rows:
            size = min(65536, pos)
            pos -= size
            f.seek(pos)
            block = f.read(size)
            blocks.append(block)
            count += block.count(b"\n")
        truncated = pos > 0
        raw = b"".join(reversed(blocks)).decode("utf-8", errors="ignore")
        lines = raw.splitlines()
        if lines and lines[0] == header:

```

```

        lines = lines[1:]
    elif truncated and lines:
        lines = lines[1:]
    lines = lines[-rows:]
    text = header + "\n" + "\n".join(lines)
    return pd.read_csv(io.StringIO(text))

def score_event(model: dict, seg: pd.DataFrame, env: np.ndarray) -> dict:
    config = model["config"]
    vector, peak, rms, freq, power = extract_one(seg, config["fs"], config["top"],
config["bands"])
    x = model_features(model, vector)
    env_filled, _ = fill_env(env.reshape(1, -1), np.asarray(model["env_medians"]))
    if model["env_model"] is not None:
        x = x - model["env_model"].predict(env_filled)
    scaled = model["scaler"].transform(x)
    recon = model["autoencoder"].predict(scaled)
    per_feature = (recon - scaled) ** 2
    score = float(np.mean(per_feature))
    threshold = float(model["threshold"])
    band_err = per_feature[0, 5:8]
    total = float(np.sum(band_err))
    bands = (band_err / total).tolist() if total > 0 else [0.0, 0.0, 0.0]
    return {"score": score, "score_ratio": score / threshold if threshold > 0 else
math.nan, "peak_hz": peak, "rms": rms, "freq_x": freq.tolist(), "freq_y":
power.tolist(), "bands": bands}

class Monitor:
    def __init__(self, bridge: Path, model_path: Path):
        self.bridge = bridge
        self.tap_path, _ = companion_paths(bridge)
        self.model = joblib.load(model_path)
        self.lock = threading.Lock()
        self.state = {"status_text": "대기 중", "status_class": "waiting",
"score_ratio": None, "peak_hz": None, "rms": None, "temp_c": None, "humidity":
None, "tap_index": None, "evaluated_count": 0, "anomaly_count": 0, "wave_x": [],
"wave_y": [], "freq_x": [], "freq_y": [], "bands": [0, 0, 0], "updated_at": None,
"info": "로거 데이터를 기다리는 중"}
        self.last_tap = -1
        self.evaluated_count = 0
        self.anomaly_count = 0
        threading.Thread(target=self.loop, daemon=True).start()

    def loop(self) -> None:
        while True:
            try:
                if not self.bridge.exists() or not self.tap_path.exists():
                    time.sleep(0.5)
                    continue
                config = self.model["config"]
                rows = int(config["fs"] * 15)
                data = tail_dataframe(self.bridge, rows)
                taps = read_taps(self.tap_path)
                if data.empty:
                    time.sleep(0.5)
                    continue
                t = pd.to_numeric(data["t_perf_ns"],
errors="coerce").to_numpy(float)
                z = pd.to_numeric(data["az_g"], errors="coerce").to_numpy(float)
                max_t = np.nanmax(t)
                recent_start = max_t - 2e9
                recent = t >= recent_start

```

```

        wave_y = z[recent]
        wave_x = ((t[recent] - t[recent][0]) / 1e9).tolist() if
np.any(recent) else []
        temp = float(pd.to_numeric(data["temp_c"],
errors="coerce").dropna().iloc[-1]) if "temp_c" in data and
pd.to_numeric(data["temp_c"], errors="coerce").notna().any() else None
        hum = float(pd.to_numeric(data["humidity"],
errors="coerce").dropna().iloc[-1]) if "humidity" in data and
pd.to_numeric(data["humidity"], errors="coerce").notna().any() else None
        unix_values = pd.to_numeric(data["t_unix_s"],
errors="coerce").dropna() if "t_unix_s" in data else pd.Series(dtype=float)
        updated_at = float(unix_values.iloc[-1]) if len(unix_values) else
time.time()
        update = {"wave_x": wave_x, "wave_y": wave_y.tolist(), "temp_c":
temp, "humidity": hum, "updated_at": updated_at}
        end_needed = taps["tap_ns"] + (self.model["delay"] +
self.model["window"]) * 1e9
        complete = taps[end_needed <= max_t]
        if len(complete):
            tap_idx = int(complete.index[-1])
            if tap_idx != self.last_tap:
                tap_ns = float(complete.iloc[-1]["tap_ns"])
                start = tap_ns + self.model["delay"] * 1e9
                end = start + self.model["window"] * 1e9
                m = (t >= start) & (t <= end)
                seg = data.loc[m]
                if len(seg) >= int(config["fs"] * self.model["window"] *
0.7):
                    env = np.array([float(pd.to_numeric(seg["temp_c"],
errors="coerce").mean()) if "temp_c" in seg else math.nan,
float(pd.to_numeric(seg["humidity"], errors="coerce").mean()) if "humidity" in seg
else math.nan])
                    result = score_event(self.model, seg, env)
                    ratio = result["score_ratio"]
                    if not np.isfinite(ratio):
                        text, css = "측정 대기", "waiting"
                    elif ratio < 0.8:
                        text, css = "정상 범위", "normal"
                    elif ratio < 1.0:
                        text, css = "주의", "warning"
                    else:
                        text, css = "이상 후보", "anomaly"
                    self.evaluated_count += 1
                    if np.isfinite(ratio) and ratio >= 1.0:
                        self.anomaly_count += 1
                    env_text = "환경 보정 사용" if
self.model.get("environment_enabled", self.model.get("env_model") is not None)
else "환경 보정 미사용"
                    update.update(result)
                    update.update({"status_text": text, "status_class":
css, "tap_index": tap_idx, "evaluated_count": self.evaluated_count,
"anomaly_count": self.anomaly_count, "info": f"Autoencoder 8-8-3-8-8 | 로그·대역비율
피쳐 | {env_text} | 분석 대역 0-{config['top']:.1f} Hz | 임계값
{self.model['threshold']:.5f} | 최근 타격 {tap_idx}"})
                    self.last_tap = tap_idx
                with self.lock:
                    self.state.update(update)
            except Exception as exc:
                with self.lock:
                    self.state["info"] = f"대기/오류: {exc}"
        time.sleep(0.5)

def get(self) -> dict:
    with self.lock:

```

```

        return dict(self.state)

def local_ip() -> str:
    try:
        result = subprocess.run(["hostname", "-I"], capture_output=True, text=True,
check=False)
        for value in result.stdout.split():
            if ":" not in value and not value.startswith("127."):
                return value
    except OSError:
        pass
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.connect(("10.255.255.255", 1))
        value = sock.getsockname()[0]
        sock.close()
        return value
    except OSError:
        return "라즈베리파이_IP"

def serve(bridge: Path, model: Path, host: str, port: int) -> None:
    monitor = Monitor(bridge, model)
    app = Flask(__name__)

    @app.get("/")
    def index():
        return HTML

    @app.get("/api")
    def api():
        return jsonify(monitor.get())

    print(f"대시보드: http://{local_ip()}:{port}")
    app.run(host=host, port=port, threaded=True, use_reloader=False)

def run(args: argparse.Namespace) -> None:
    output = Path(args.output_dir)
    output.mkdir(parents=True, exist_ok=True)
    bridge = output / f"{args.condition}_bridge_log.csv"
    tap_path, meta_path = companion_paths(bridge)
    for path in (bridge, tap_path, meta_path):
        if path.exists():
            path.unlink()
    logger = Path(args.logger)
    if not logger.is_absolute() and not logger.exists():
        candidate = Path(__file__).resolve().parent / logger
        if candidate.exists():
            logger = candidate
    model = Path(args.model)
    if not model.exists():
        raise FileNotFoundError(f"모델 파일을 찾지 못했습니다: {model}")
    command = [sys.executable, str(logger), "--condition", args.condition,
"--output-dir", str(output), "--sample-rate", str(args.sample_rate), "--range-g",
str(args.range_g), "--pulse-ms", str(args.pulse_ms), "--period-s",
str(args.period_s), "--warmup", str(args.warmup), "--n-taps", str(args.n_taps)]
    process = subprocess.Popen(command)
    try:
        deadline = time.time() + args.warmup * args.period_s + 30
        while not bridge.exists() and time.time() < deadline:
            if process.poll() is not None:
                raise RuntimeError("로거가 먼저 종료되었습니다.")

```

```

        time.sleep(0.5)
    if not bridge.exists():
        raise RuntimeError("bridge CSV가 생성되지 않았습니다.")
    serve(bridge, model, args.host, args.port)
finally:
    if process.poll() is None:
        process.terminate()

def parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser()
    sub = p.add_subparsers(dest="mode", required=True)
    t = sub.add_parser("train")
    t.add_argument("--bridge", nargs="+", required=True)
    t.add_argument("--model", default="sentry_autoencoder.joblib")
    t.add_argument("--delay", type=float, default=0.05)
    t.add_argument("--window", type=float, default=2.0)
    t.add_argument("--train-count", type=int, default=120)
    t.add_argument("--threshold-count", type=int, default=30)
    t.add_argument("--test-count", type=int, default=30)
    t.add_argument("--seed", type=int, default=42)
    e = sub.add_parser("evaluate")
    e.add_argument("--model", default="sentry_autoencoder.joblib")
    e.add_argument("--normal", nargs="*", default=[])
    e.add_argument("--anomaly", nargs="*", default=[])
    e.add_argument("--output", default="evaluation_summary.csv")
    d = sub.add_parser("dashboard")
    d.add_argument("--bridge", required=True)
    d.add_argument("--model", default="sentry_autoencoder.joblib")
    d.add_argument("--host", default="0.0.0.0")
    d.add_argument("--port", type=int, default=8080)
    r = sub.add_parser("run")
    r.add_argument("--logger", default="logger3_fixed.py")
    r.add_argument("--model", default="sentry_autoencoder.joblib")
    r.add_argument("--condition", default="T0-H0-M0-B0")
    r.add_argument("--output-dir", default="data/live")
    r.add_argument("--sample-rate", type=int, default=500)
    r.add_argument("--range-g", type=int, default=8)
    r.add_argument("--pulse-ms", type=float, default=40)
    r.add_argument("--period-s", type=float, default=5)
    r.add_argument("--warmup", type=int, default=10)
    r.add_argument("--n-taps", type=int, default=35)
    r.add_argument("--host", default="0.0.0.0")
    r.add_argument("--port", type=int, default=8080)
    return p

def main() -> None:
    args = parser().parse_args()
    if args.mode == "train":
        train(args)
    elif args.mode == "evaluate":
        evaluate(args)
    elif args.mode == "dashboard":
        serve(Path(args.bridge), Path(args.model), args.host, args.port)
    else:
        run(args)

if __name__ == "__main__":
    main()

```

logger3_fixed.py

```
from __future__ import annotations

import argparse
import csv
import json
import math
import multiprocessing as mp
import queue
import threading
import time
from pathlib import Path
from typing import Final

import adafruit_dht
import board
from gpiozero import OutputDevice
from smbus2 import SMBus

ADXL355_ADDR_LOW: Final = 0x1D
ADXL355_ADDR_HIGH: Final = 0x53

REG_DEVID_AD: Final = 0x00
REG_TEMP2: Final = 0x06
REG_XDATA3: Final = 0x08
REG_FILTER: Final = 0x28
REG_RANGE: Final = 0x2C
REG_POWER_CTL: Final = 0x2D
REG_RESET: Final = 0x2F

EXPECTED_ID: Final = (0xAD, 0x1D, 0xED)

RANGE_TO_CODE = {2: 0x01, 4: 0x02, 8: 0x03}
RANGE_TO_SCALE_G_PER_LSB = {
    2: 2.048 / (2**19),
    4: 4.096 / (2**19),
    8: 8.192 / (2**19),
}
RANGE_TO_FULL_SCALE_G = {2: 2.048, 4: 4.096, 8: 8.192}

SAMPLE_RATE_TO_FILTER = {125: 0x05, 250: 0x04, 500: 0x03}

TEMP_BIAS_LSB: Final = 1852.0
TEMP_SLOPE_LSB_PER_C: Final = -9.05

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="SENTRY ADXL355 + DHT22 + solenoid tapper logger"
    )
    parser.add_argument(
        "--condition",
        default="Tx-Hx-M0-B0",
        help="조건 코드. 예: T0-H0-M500-B0",
    )
    parser.add_argument("--output-dir", default="data", help="CSV 저장 폴더")
    parser.add_argument("--bus", type=int, default=1, help="I2C 버스 번호")
    parser.add_argument(
        "--address",
        choices=["auto", "0x1d", "0x53"],
    )
```

```

        default="auto",
        help="ADXL355 I2C 주소",
    )
    parser.add_argument(
        "--sample-rate",
        type=int,
        choices=sorted(SAMPLE_RATE_TO_FILTER),
        default=500,
        help="ADXL355 ODR와 로거 목표 샘플링 주파수. 최종 실험 전체에서 고정",
    )
    parser.add_argument(
        "--range-g",
        type=int,
        choices=[2, 4, 8],
        default=8,
        help="가속도 측정 범위",
    )
    parser.add_argument(
        "--dht-interval",
        type=float,
        default=2.0,
        help="DHT22 측정 간격(초)",
    )
    parser.add_argument(
        "--adxl-temp-interval",
        type=float,
        default=1.0,
        help="ADXL355 내부 온도 측정 간격(초)",
    )
    parser.add_argument(
        "--pulse-ms",
        type=float,
        default=40.0,
        help="태퍼 펄스폭(ms). 예비 실험에서 포화가 없도록 확정",
    )
    parser.add_argument(
        "--period-s",
        type=float,
        default=5.0,
        help="타격 시작 시점 간 주기(초)",
    )
    parser.add_argument(
        "--warmup",
        type=int,
        default=10,
        help="기록 전에 같은 펄스·같은 주기로 수행할 워밍업 타격 횟수",
    )
    parser.add_argument(
        "--pre-tap-seconds",
        type=float,
        default=2.0,
        help="기록 시작 후 첫 본 타격까지의 대기 시간",
    )
    parser.add_argument("--n-taps", type=int, default=35, help="본 타격 횟수")
    parser.add_argument(
        "--tail-seconds",
        type=float,
        default=5.0,
        help="마지막 타격 후 추가 수집 시간",
    )
    parser.add_argument(
        "--flush-seconds",
        type=float,
        default=1.0,

```

```

        help="CSV 디스크 flush 간격",
    )
    parser.add_argument(
        "--tapper-gpio",
        type=int,
        default=17,
        help="솔레노이드 드라이버 제어 BCM GPIO",
    )
    parser.add_argument(
        "--no-tapper",
        action="store_true",
        help="테퍼 없이 연속 수집. Ctrl+C로 종료",
    )
    parser.add_argument(
        "--no-dht",
        action="store_true",
        help="DHT22 없이 가속도만 수집",
    )
    return parser.parse_args()

def twos_complement(value: int, bits: int) -> int:
    sign_bit = 1 << (bits - 1)
    return value - (1 << bits) if value & sign_bit else value

def detect_address(bus: SMBus, requested: str) -> int:
    candidates = (
        [ADXL355_ADDR_LOW, ADXL355_ADDR_HIGH]
        if requested == "auto"
        else [int(requested, 16)]
    )
    for address in candidates:
        try:
            device_id = tuple(bus.read_i2c_block_data(address, REG_DEVID_AD, 3))
        except OSError:
            continue
        if device_id == EXPECTED_ID:
            return address
    raise RuntimeError(
        "ADXL355를 찾지 못했습니다. i2cdetect -y 1에서 1d 또는 53이 "
        "보이는지 확인하세요."
    )

def configure_adxl355(
    bus: SMBus, address: int, range_g: int, sample_rate: int
) -> None:
    bus.write_byte_data(address, REG_RESET, 0x52)
    time.sleep(0.1)

    bus.write_byte_data(address, REG_RANGE, 0x80 | RANGE_TO_CODE[range_g])
    bus.write_byte_data(address, REG_FILTER, SAMPLE_RATE_TO_FILTER[sample_rate])
    bus.write_byte_data(address, REG_POWER_CTL, 0x00)
    time.sleep(0.1)

def read_adxl_temperature_c(bus: SMBus, address: int) -> float:
    temp2, temp1 = bus.read_i2c_block_data(address, REG_TEMP2, 2)
    raw = ((temp2 & 0x0F) << 8) | temp1
    return 25.0 + (raw - TEMP_BIAS_LSB) / TEMP_SLOPE_LSB_PER_C

```

```

def read_acceleration_g(
    bus: SMBus, address: int, scale_g_per_lsb: float
) -> tuple[float, float, float]:
    data = bus.read_i2c_block_data(address, REG_XDATA3, 9)
    values: list[float] = []
    for i in range(0, 9, 3):
        raw20 = (data[i] << 12) | (data[i + 1] << 4) | (data[i + 2] >> 4)
        values.append(twos_complement(raw20, 20) * scale_g_per_lsb)
    return values[0], values[1], values[2]

def wait_until_ns(target_ns: int) -> None:
    while True:
        remaining_ns = target_ns - time.perf_counter_ns()
        if remaining_ns <= 0:
            return
        if remaining_ns > 2_000_000:
            time.sleep((remaining_ns - 1_000_000) / 1e9)
        else:
            time.sleep(0)

def pulse_tapper(tap: OutputDevice, pulse_seconds: float) -> tuple[int, int]:
    start_ns = time.perf_counter_ns()
    tap.on()
    time.sleep(pulse_seconds)
    tap.off()
    end_ns = time.perf_counter_ns()
    return start_ns, end_ns

def run_warmup(
    tap: OutputDevice,
    warmup_count: int,
    period_s: float,
    pulse_s: float,
) -> None:
    if warmup_count <= 0:
        return
    print(
        f"[태퍼] 워밍업 {warmup_count}회: 본 실험과 같은 "
        f"{period_s:.3f} s 주기, {pulse_s * 1000:.1f} ms 펄스"
    )
    start_ns = time.perf_counter_ns() + 1_000_000_000
    period_ns = int(period_s * 1e9)
    for i in range(warmup_count):
        wait_until_ns(start_ns + i * period_ns)
        pulse_tapper(tap, pulse_s)

def unix_from_perf_ns(
    perf_ns: int, anchor_perf_ns: int, anchor_unix_s: float
) -> float:
    return anchor_unix_s + (perf_ns - anchor_perf_ns) / 1e9

def dht_worker(
    output_queue: mp.Queue,
    stop_event: mp.Event,
    interval_s: float,
) -> None:
    dht = adafruit_dht.DHT22(board.D4)
    try:

```

```

while not stop_event.is_set():
    measured_ns = time.perf_counter_ns()
    temp_c = math.nan
    humidity = math.nan
    try:
        temp = dht.temperature
        hum = dht.humidity
        if temp is not None:
            temp_c = float(temp)
        if hum is not None:
            humidity = float(hum)
    except RuntimeError:
        pass

    item = (measured_ns, temp_c, humidity)
    try:
        while True:
            output_queue.get_nowait()
    except queue.Empty:
        pass
    try:
        output_queue.put_nowait(item)
    except queue.Full:
        pass

    stop_event.wait(interval_s)
finally:
    dht.exit()

```

```

def tapper_worker(
    args: argparse.Namespace,
    tap: OutputDevice,
    tap_path: Path,
    stop_event: threading.Event,
    anchor_perf_ns: int,
    anchor_unix_s: float,
    first_tap_ns: int,
) -> None:
    pulse_s = args.pulse_ms / 1000.0
    period_ns = int(args.period_s * 1e9)

    try:
        with tap_path.open("w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(
                [
                    "tap_index",
                    "scheduled_perf_ns",
                    "pulse_start_perf_ns",
                    "pulse_end_perf_ns",
                    "pulse_start_unix_s",
                    "start_delay_ms",
                    "condition",
                ]
            )

        print(
            f"[태퍼] 본 타격 {args.n_taps}회 시작 "
            f"({args.period_s:.3f} s start-to-start)"
        )
        for index in range(args.n_taps):
            scheduled_ns = first_tap_ns + index * period_ns
            wait_until_ns(scheduled_ns)

```

```

        start_ns, end_ns = pulse_tapper(tap, pulse_s)
        writer.writerow(
            [
                index,
                scheduled_ns,
                start_ns,
                end_ns,
                f"{unix_from_perf_ns(start_ns, anchor_perf_ns,
anchor_unix_s):.9f}",
                f"((start_ns - scheduled_ns) / 1e6:.3f)",
                args.condition,
            ]
        )
        file.flush()

    print(
        f"[태퍼] 완료. 마지막 타격 후 {args.tail_seconds:.2f} s 추가 수집"
    )
    time.sleep(args.tail_seconds)
finally:
    stop_event.set()

def format_optional(value: float, digits: int = 3) -> str:
    return "" if math.isnan(value) else f"{value:.{digits}f}"

def main() -> None:
    args = parse_args()
    if args.period_s <= args.pulse_ms / 1000.0:
        raise ValueError("period-s는 pulse-ms보다 길어야 합니다.")

    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    accel_path = output_dir / f"{args.condition}_bridge_log.csv"
    tap_path = output_dir / f"{args.condition}_tap_events.csv"
    metadata_path = output_dir / f"{args.condition}_metadata.json"

    scale = RANGE_TO_SCALE_G_PER_LSB[args.range_g]
    full_scale_g = RANGE_TO_FULL_SCALE_G[args.range_g]
    clip_limit_g = 0.995 * full_scale_g
    sample_interval_ns = int(1e9 / args.sample_rate)

    tap: OutputDevice | None = None
    tapper_thread: threading.Thread | None = None
    dht_process: mp.Process | None = None
    dht_queue: mp.Queue | None = None
    dht_stop: mp.Event | None = None
    stop_event = threading.Event()

    latest_dht_temp = math.nan
    latest_humidity = math.nan
    latest_dht_perf_ns = 0
    latest_adxl_temp = math.nan
    last_adxl_temp_ns = 0

    total_samples = 0
    missed_slots = 0
    clip_counts = {"ax": 0, "ay": 0, "az": 0}

    try:
        if not args.no_tapper:
            tap = OutputDevice(

```

```

        args.tapper_gpio, active_high=True, initial_value=False
    )

    if not args.no_dht:
        ctx = mp.get_context("spawn")
        dht_queue = ctx.Queue(maxsize=1)
        dht_stop = ctx.Event()
        dht_process = ctx.Process(
            target=dht_worker,
            args=(dht_queue, dht_stop, args.dht_interval),
            daemon=True,
        )
        dht_process.start()

    with SMBus(args.bus) as bus:
        address = detect_address(bus, args.address)
        configure_adxl355(
            bus, address, args.range_g, args.sample_rate
        )

    if not args.no_tapper:
        if tap is None:
            raise RuntimeError("테퍼 GPIO 초기화에 실패했습니다.")
        run_warmup(
            tap,
            args.warmup,
            args.period_s,
            args.pulse_ms / 1000.0,
        )

    anchor_perf_ns = time.perf_counter_ns()
    anchor_unix_s = time.time()
    first_tap_ns = (
        anchor_perf_ns + int(args.pre_tap_seconds * 1e9)
    )

    metadata = {
        "condition": args.condition,
        "address_hex": f"0x{address:02X}",
        "sample_rate_hz": args.sample_rate,
        "filter_register_hex": (
            f"0x{SAMPLE_RATE_TO_FILTER[args.sample_rate]:02X}"
        ),
        "range_g": args.range_g,
        "pulse_ms": args.pulse_ms,
        "period_s": args.period_s,
        "warmup_taps": args.warmup,
        "recorded_taps": 0 if args.no_tapper else args.n_taps,
        "pre_tap_seconds": args.pre_tap_seconds,
        "tail_seconds": args.tail_seconds,
        "start_unix_s": anchor_unix_s,
    }

    metadata_path.write_text(
        json.dumps(metadata, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

    print(f"[수집] 조건: {args.condition}")
    print(
        f"[수집] ADXL355 0x{address:02X}, "
        f"±{args.range_g} g, {args.sample_rate} Hz"
    )

    print(f"[수집] 가속도 CSV: {accel_path}")
    if not args.no_tapper:

```

```

        print(f"[수집] 타격 CSV: {tap_path}")

    if not args.no_tapper:
        tapper_thread = threading.Thread(
            target=tapper_worker,
            args=(
                args,
                tap,
                tap_path,
                stop_event,
                anchor_perf_ns,
                anchor_unix_s,
                first_tap_ns,
            ),
            daemon=True,
        )
        tapper_thread.start()

    next_sample_ns = time.perf_counter_ns()
    next_flush_ns = (
        next_sample_ns + int(args.flush_seconds * 1e9)
    )

    with accel_path.open(
        "w", newline="", encoding="utf-8", buffering=1024 * 1024
    ) as file:
        writer = csv.writer(file)
        writer.writerow(
            [
                "sample_index",
                "t_perf_ns",
                "t_unix_s",
                "scheduled_perf_ns",
                "lateness_us",
                "missed_slots_total",
                "temp_c",
                "humidity",
                "dht_age_s",
                "adxl_temp_c",
                "ax_g",
                "ay_g",
                "az_g",
            ]
        )

    while not stop_event.is_set():
        wait_until_ns(next_sample_ns)
        now_ns = time.perf_counter_ns()

        if now_ns - next_sample_ns >= sample_interval_ns:
            skipped = int(
                (now_ns - next_sample_ns) // sample_interval_ns
            )
            missed_slots += skipped
            next_sample_ns += skipped * sample_interval_ns

        scheduled_ns = next_sample_ns

        if dht_queue is not None:
            try:
                while True:
                    (
                        latest_dht_perf_ns,

```

```

        temp_value,
        humidity_value,
    ) = dht_queue.get_nowait()
    if not math.isnan(temp_value):
        latest_dht_temp = temp_value
    if not math.isnan(humidity_value):
        latest_humidity = humidity_value
except queue.Empty:
    pass

if (
    now_ns - last_adxl_temp_ns
    >= int(args.adxl_temp_interval * 1e9)
):
    try:
        latest_adxl_temp = read_adxl_temperature_c(
            bus, address
        )
    except OSError:
        latest_adxl_temp = math.nan
    last_adxl_temp_ns = now_ns

read_start_ns = time.perf_counter_ns()
ax_g, ay_g, az_g = read_acceleration_g(
    bus, address, scale
)
read_end_ns = time.perf_counter_ns()
sample_perf_ns = (read_start_ns + read_end_ns) // 2

for key, value in (
    ("ax", ax_g),
    ("ay", ay_g),
    ("az", az_g),
):
    if abs(value) >= clip_limit_g:
        clip_counts[key] += 1

dht_age_s = (
    math.nan
    if latest_dht_perf_ns == 0
    else (sample_perf_ns - latest_dht_perf_ns) / 1e9
)

writer.writerow(
    [
        total_samples,
        sample_perf_ns,
        f"{unix_from_perf_ns(sample_perf_ns, anchor_perf_ns,
anchor_unix_s):.9f}",
        scheduled_ns,
        f"{(sample_perf_ns - scheduled_ns) / 1000.0:.3f}",
        missed_slots,
        format_optional(latest_dht_temp),
        format_optional(latest_humidity),
        format_optional(dht_age_s, 3),
        format_optional(latest_adxl_temp),
        f"{ax_g:.8f}",
        f"{ay_g:.8f}",
        f"{az_g:.8f}",
    ]
)
total_samples += 1
next_sample_ns += sample_interval_ns

```

```

        if read_end_ns >= next_flush_ns:
            file.flush()
            next_flush_ns = (
                read_end_ns + int(args.flush_seconds * 1e9)
            )

except KeyboardInterrupt:
    print("\n[수집] Ctrl+C로 중단됨")
finally:
    stop_event.set()

    if tapper_thread is not None:
        tapper_thread.join(timeout=3.0)

    if dht_stop is not None:
        dht_stop.set()
    if dht_process is not None:
        dht_process.join(timeout=3.0)
        if dht_process.is_alive():
            dht_process.terminate()

    if tap is not None:
        tap.off()
        tap.close()

    print("[수집] 종료")
    print(f"[QC] 저장 샘플: {total_samples}")
    print(f"[QC] 건너편 샘플 슬롯: {missed_slots}")
    print(
        "[QC] 포화 샘플 수: "
        f"ax={clip_counts['ax']}, "
        f"ay={clip_counts['ay']}, "
        f"az={clip_counts['az']}"
    )
    if any(clip_counts.values()):
        print(
            "[경고] 포화가 발생했습니다. pulse-ms를 줄이거나 "
            "range-g를 높인 뒤 재수집하세요."
        )

if __name__ == "__main__":
    main()

```